

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)»**

Компьютерные науки и прикладная математика

УДК 519.6

СОГЛАСОВАНО

Научный руководитель,
Доктор физико-математических
наук, доцент

_____ А. Ю. Морозов
« ____ » _____ 2026 г.

УТВЕРЖДАЮ

Руководитель
образовательной программы

_____ С. С. Крылов
« ____ » _____ 2026 г.

**Выпускная квалификационная работа
(академическая)**

на тему: **Реализация параллельных алгоритмов интерполяции на основе
разреженных сеток с квадратичным базисом**

по направлению подготовки 02.03.02 «Фундаментальная информатика и
информационные технологии»

ВЫПОЛНИЛ

студент группы М8О-411Б-22
образовательной программы
02.03.02 «Фундаментальная
информатика и
информационные технологии»

_____ И. Н. Ивкович
« ____ » _____ 2026 г.

РЕФЕРАТ

Итоговая аттестационная работа состоит из 52 страниц, 4 глав, 1 рисунка, 7 листингов, 1 приложения. Использовано 10 источников.

РАЗРЕЖЕННЫЕ СЕТКИ; ИНТЕРПОЛЯЦИЯ; КВАДРАТИЧНЫЙ БАЗИС; ПАРАЛЛЕЛЬНЫЕ АЛГОРИТМЫ; ЧИСЛЕННЫЕ МЕТОДЫ; ДИФФЕРЕНЦИАЛЬНЫЕ УРАВНЕНИЯ; GO; FLUTTER

Объект разработки: программный комплекс для интерполяции многомерных отображений и решений дифференциальных уравнений на основе адаптивных разреженных сеток.

Цель работы: реализация и исследование параллельных алгоритмов интерполяции на основе разреженных сеток с квадратичным базисом, а также создание прикладной системы для удалённого вычисления и визуализации результатов.

Методы и методология: методы численного анализа и интерполяции; иерархические базисные функции; адаптивное уточнение по коэффициентам избытка; использование контрольных точек; численное интегрирование систем обыкновенных дифференциальных уравнений методом Рунге–Кутты–Фельберга; методы параллельного программирования; клиент-серверная архитектура с использованием gRPC и protobuf.

Результаты работы и их новизна: разработано вычислительное ядро на языке Go, реализующее построение адаптивной разреженной сетки для скалярных и векторных отображений в последовательном и параллельном режимах. Реализована поддержка квадратичного базиса, позволяющего получать более гладкую аппроксимацию по сравнению с линейным вариантом. Создан серверный слой удалённого доступа, интерпретирующий пользовательские формулы и поддерживающий аппроксимацию операторов, возникающих при решении дифференциальных уравнений. Разработано клиентское приложение визуализации на Flutter. Научная новизна состоит в объединении адаптивной разреженной интерполяции, параллельного построения и прикладной инфраструктуры визуального анализа в рамках единого программного комплекса.

Область применения: визуализация и исследование двумерных динамических систем; численный эксперимент с операторами перехода; демонстрация методов адаптивной интерполяции в учебных курсах; использование как основы для дальнейших исследований в области параллельных численных методов.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ВВЕДЕНИЕ	8
1 АРХИТЕКТУРА ПРОГРАММНОГО КОМПЛЕКСА И МАТЕМАТИЧЕСКИЕ ОСНОВЫ МЕТОДА РАЗРЕЖЕННЫХ СЕТОК	11
1.1 Интерполяция многомерных функций и проблема роста вычислительной сложности	11
1.2 Нормировка области и постановка задачи интерполяции	11
1.3 Одномерные иерархические базисные функции	12
1.4 Многомерный базис и тензорное произведение	13
1.5 Иерархическое представление и коэффициенты избытка	13
1.6 Адаптивное уточнение и локальный критерий продолжения	13
1.7 Проблема вырожденного критерия на примере $\sin(x)$	14
1.8 Механизм якорных точек как расширение базового метода	14
1.9 Порождение дочерних узлов	15
1.10 Жёсткое ограничение числа узлов как механизм оптимизации	15
1.11 Рекурсивное вычисление интерполянта	15
1.12 Связь с задачами моделирования динамических систем	16
Выводы по главе	16
2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И СТРУКТУР ДАННЫХ ПРОГРАММНОГО КОМПЛЕКСА	17
2.1 Роль вычислительного модуля в структуре проекта	17
2.2 Требования к вычислительному модулю	17
2.3 Центральный тип AdaptiveSparseGrid	18
2.4 Проектирование базовых структур данных	18
2.5 Архитектура построения сетки	19
2.6 Архитектура поддержки parallel и sequential build	19
2.7 Поддержка якорных точек как архитектурное расширение	19
2.8 Жёсткое ограничение числа узлов как эксплуатационный параметр	20
2.9 Композиция отображений и метод следующей итерации	20
2.10 Поддержка дифференциальных уравнений на уровне архитектуры	20
2.11 Вспомогательный серверный слой	21
2.12 Клиентская визуализация как вспомогательный инструмент	21
2.13 Наблюдаемость и эксплуатационные характеристики	21
Выводы по главе	22

3 РЕАЛИЗАЦИЯ ПАРАЛЛЕЛЬНЫХ АЛГОРИТМОВ, АНАЛИЗ РЕЗУЛЬТАТОВ И ПОДГОТОВКА ИЛЛЮСТРАЦИЙ	23
3.1 Реализация вычислительного ядра на Go	23
3.2 Ключи узлов и кодирование положения в сетке	24
3.3 Реализация базисных функций	25
3.4 Параллельный и последовательный режимы построения	26
3.5 Создание узла и вычисление коэффициента избытка	27
3.6 Ключевой участок buildGrid: адаптация, якорные точки и ограничение числа узлов	28
3.7 Рекурсивная интерполяция по построенной сетке	29
3.8 Два режима работы с дифференциальными уравнениями	30
3.9 Вспомогательный серверный слой	31
3.10 Клиентская визуализация	31
3.11 Анализ структуры бенчмарков	31
3.12 Анализ полученных данных	31
Выводы по главе	32
4 ОБОБЩЕНИЕ РЕЗУЛЬТАТОВ И НАПРАВЛЕНИЯ ДАЛЬНЕЙ- ШЕГО РАЗВИТИЯ	33
4.1 Соответствие результатов поставленным задачам	33
4.2 Общая характеристика экспериментальной программы	33
4.3 Зависимость времени построения от размерности и размера области в Go- реализации	34
4.4 Сравнение последовательного и параллельного построения в Go-реализации	35
4.5 Сравнение линейного и квадратичного базисов в Go-реализации	36
4.6 Влияние якорных точек на производительность и корректность построения	38
4.7 Два подхода к работе с дифференциальными уравнениями в Go-реализации	39
4.8 Жёсткое ограничение числа узлов как средство оптимизации	40
4.9 Стоимость вычисления значения после построения	41
4.10 Четырёхмерная функция: сравнение всех сочетаний BuildType и BasisType	42
4.11 Сравнение Go- и C++-реализаций библиотеки	43
4.11.1 Сравнение последовательных реализаций	43
4.11.2 Сравнение параллельных реализаций	44
4.11.3 Сравнение Go- и C++-реализаций на задаче Коши	44
4.11.4 Сравнение ограничения числа узлов	45
4.12 Преимущества разработанного решения	45
4.13 Ограничения текущей реализации	46
4.14 Иллюстрации двумерных функций, полученные во Flutter-клиенте	46
4.15 Перспективы дальнейшего развития	48
Выводы по главе	48

ЗАКЛЮЧЕНИЕ	50
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	52

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Flutter – кроссплатформенный UI-фреймворк, использованный для создания клиентского приложения визуализации

Go – компилируемый язык программирования, использованный в работе для реализации вычислительного ядра и серверного слоя

gRPC – фреймворк удалённого вызова процедур, использующий protobuf-контракты и HTTP/2 для обмена структурированными сообщениями между клиентом и сервером

Адаптивное уточнение – процесс локального увеличения детализации разреженной сетки в тех областях, где текущая аппроксимация оказывается недостаточно точной

Базисная функция – локальная функция, связанная с узлом сетки и определяющая вклад этого узла в итоговый интерполянт

Векторное поле – отображение, сопоставляющее каждой точке области вектор, например скорость изменения состояния динамической системы

Входная точка сетки (Entry Point) – начальный узел или стартовая конфигурация уровней и индексов, от которой начинается локальное построение адаптивной ветви разреженной сетки

Дифференциальное уравнение – уравнение, связывающее неизвестную функцию и её производные; в работе используется для задания динамики системы через правую часть

Иерархический коэффициент (коэффициент избытка) – разность между точным значением функции в узле и значением, восстановленным по уже построенной части сетки

Интерполянт – приближённое представление функции, позволяющее вычислять её значения по заранее построенной системе узлов и коэффициентов

Квадратичный базис – вариант базисной функции, в котором локальный вклад зависит квадратично от расстояния до центра узла; используется в работе как основной режим аппроксимации

Параллельный алгоритм – алгоритм, в котором независимые части вычислений выполняются одновременно на нескольких вычислительных потоках или ядрах процессора

Разреженная сетка (Sparse Grid) – иерархическая система узлов, предназначенная для аппроксимации многомерных функций с меньшим числом точек по сравнению с полной тензорной сеткой

Рекурсивный обход – способ обработки иерархической структуры данных, при котором вычисление в текущем узле дополняется вычислением во всех релевантных дочерних узлах

Тензорная сетка – полная многомерная сетка, получаемая декартовым произведением одномерных сеток; характеризуется быстрым ростом числа узлов с увеличением размерности

Численное интегрирование – приближённое вычисление решения дифференциального уравнения на конечном интервале времени с помощью дискретной схемы

Якорная точка (Anchor Point) – заранее заданная точка области, в которой дополнительно проверяется ошибка интерполяции для принятия решения о продолжении адаптивного уточнения

ВВЕДЕНИЕ

Актуальность темы исследования

Во многих задачах вычислительной математики и моделирования требуется многократно вычислять значения сложных отображений на некоторой области параметров. Если исходная функция задаётся аналитически, но содержит дорогостоящие операции, либо определяется как результат численного интегрирования системы дифференциальных уравнений, прямой пересчёт в каждой точке становится вычислительно затратным. Особенно это проявляется в задачах визуализации фазовых портретов, исследования динамических систем, анализа векторных полей и построения операторов эволюции.

Одним из эффективных способов уменьшения вычислительных затрат является интерполяция на основе разреженных сеток [1; 2]. Разреженные сетки позволяют существенно сократить число узлов по сравнению с полной тензорной дискретизацией и при этом сохранить управляемую точность. Дополнительный интерес представляет адаптивное уточнение, при котором вычислительные ресурсы концентрируются только в областях сложного поведения функции. В сочетании с параллельной обработкой независимых задач такой подход делает возможным построение эффективных вычислительных модулей для интерактивных и серверных систем.

Степень разработанности проблемы

Теория разреженных сеток и иерархических базисов достаточно хорошо развита в численном анализе [1—3]. Для построения интерполянтов традиционно используются линейные базисные функции, однако в практических задачах аппроксимации гладких отображений более интересным оказывается квадратичный базис, способный лучше описывать локальную кривизну функции. Вопросы численного решения дифференциальных уравнений, в свою очередь, тесно связаны с задачей построения операторов перехода [4; 5].

В работе Морозова А. Ю. [6], использованной как одна из основных содержательных опор при выполнении дипломного проекта, рассматривается моделирование динамических систем с интервальными параметрами. Эта постановка естественным образом подводит к необходимости строить аппроксимации отображений, зависящих от неопределённых параметров, в том числе от времени. Однако практическая задача реализации параллельного вычислительного модуля, способного работать как с обычными отображениями, так и с операторами перехода, требует отдельного архитектурного и алгоритмического исследования.

Проблема исследования

При построении интерполяции для сложных отображений возникает противоречие между точностью и стоимостью вычислений. С одной стороны, повышение точности требует увеличения числа узлов и углубления адаптивного уточнения. С другой стороны, в прикладной системе время построения и стоимость последующего вычисления интерполанта должны оставаться ограниченными. Дополнительную сложность созда-

ёт то, что базовый адаптивный критерий может ошибочно остановить построение на функциях с осцилляциями и нулевыми значениями в ключевых точках. Следовательно, требуется метод и программная реализация, обеспечивающие баланс между точностью, скоростью, управляемостью размера сетки и устойчивостью адаптивного алгоритма.

Цель работы

Целью выпускной квалификационной работы является реализация и исследование параллельных алгоритмов интерполяции на основе разреженных сеток с квадратичным базисом, а также создание вычислительного модуля, пригодного для аппроксимации как обычных отображений, так и решений дифференциальных уравнений.

Задачи исследования

Для достижения поставленной цели в работе были сформулированы и решены следующие задачи:

- 1) Исследовать математические основы иерархической интерполяции на разреженных сетках и сравнить линейный и квадратичный базисы.
- 2) Разработать структуры данных для представления адаптивной разреженной сетки, узлов, входных точек и коэффициентов избытка.
- 3) Реализовать вычислительный модуль, поддерживающий последовательный и параллельный режимы построения.
- 4) Разработать механизм якорных точек для управления уточнением в случаях, когда базовый адаптивный критерий даёт ложный результат.
- 5) Реализовать возможность жёсткого ограничения числа узлов как дополнительный механизм оптимизации.
- 6) Реализовать два способа работы с дифференциальными уравнениями: через параметр времени как интервальную неопределённость и через итеративное применение `MakeNextIteration`.
- 7) Провести бенчмаркинг и проанализировать производительность разработанного решения в различных режимах работы.

Объект и предмет исследования

Объектом исследования являются методы интерполяции многомерных отображений и операторов перехода динамических систем на основе разреженных сеток.

Предметом исследования являются параллельные алгоритмы построения адаптивных разреженных сеток с квадратичным базисом, а также связанные с ними программные структуры данных и режимы вычислений.

Методы исследования

В работе использовались методы численного анализа, теории интерполяции, математического моделирования динамических систем, а также методы проектирования программного обеспечения. Для задач динамики использовалось численное интегрирование обыкновенных дифференциальных уравнений методом Рунге–Кутты–Фельберга

[4]. Для оценки эффективности применялись модульное тестирование и бенчмаркинг вычислительного модуля.

Научная новизна и практическая значимость

Научная новизна работы состоит в реализации и исследовании параллельного вычислительного модуля для адаптивной интерполяции на основе разреженных сеток с квадратичным базисом, а также в расширении базового подхода за счёт введения механизма якорных точек, который позволяет вручную устранять ложную стабилизацию алгоритма на осциллирующих функциях. Дополнительным практически значимым результатом является реализация двух альтернативных способов работы с дифференциальными уравнениями и механизма жёсткого ограничения числа узлов.

Практическая значимость работы заключается в создании программного комплекса, ядром которого является модуль `compute`, пригодный для исследования динамических систем, численного эксперимента и визуализации структуры адаптивной сетки. Серверный слой и клиент визуализации расширяют область применения разработанного модуля и делают его удобным для удалённого использования.

Апробация результатов

Результаты работы не проходили апробацию в форме публикаций и докладов на конференциях.

1 АРХИТЕКТУРА ПРОГРАММНОГО КОМПЛЕКСА И МАТЕМАТИЧЕСКИЕ ОСНОВЫ МЕТОДА РАЗРЕЖЕННЫХ СЕТОК

В первой главе рассматриваются математические основы интерполяции на разреженных сетках, лежащие в основе разработанного вычислительного модуля [1; 2]. Основное внимание уделяется иерархическому представлению аппроксимируемой функции, построению линейных и квадратичных базисных функций, критериям адаптивного уточнения и связи метода с моделированием динамических систем. Отдельно рассматривается предложенный в работе механизм якорных точек, предназначенный для преодоления характерного вырожденного сценария адаптивной интерполяции.

1.1 Интерполяция многомерных функций и проблема роста вычислительной сложности

Пусть задано отображение

$$F: \Omega \subset \mathbb{R}^d \rightarrow \mathbb{R}^m, \quad (1)$$

которое требуется многократно вычислять на области $\Omega = [a_1, b_1] \times \dots \times [a_d, b_d]$. Если каждое вычисление $F(x)$ связано с численным интегрированием, моделированием динамики или интерпретацией составной формулы, возникает задача построения приближённого интерполанта

$$\tilde{F}(x) \approx F(x), \quad x \in \Omega, \quad (2)$$

который обеспечивал бы контролируемую ошибку при существенно меньшей стоимости вычисления [3; 5].

Прямое построение полной тензорной сетки в d -мерном случае требует экспоненциального роста числа узлов. Если в каждом измерении используется n узлов, то общее число точек равно n^d . Такое поведение известно как «проклятие размерности». Поэтому вместо полной сетки применяются разреженные конструкции, в которых сохраняются только наиболее информативные комбинации одномерных уровней детализации [1; 2].

1.2 Нормировка области и постановка задачи интерполяции

В разработанной системе вычисления выполняются на единичном гиперкубе. Для этого исходная точка $x = (x_1, \dots, x_d)$ переводится из реальной области Ω в нормированные координаты $u = (u_1, \dots, u_d)$ по формуле

$$u_i = \frac{x_i - a_i}{b_i - a_i}, \quad i = 1, \dots, d. \quad (3)$$

Обратное преобразование имеет вид

$$x_i = a_i + u_i(b_i - a_i). \quad (4)$$

Такое нормирование позволяет единообразно задавать базисные функции и не связывать структуру метода с конкретным физическим масштабом задачи. В результате и математическая модель, и реализация модуля `compute` опираются на одну и ту же нормированную постановку.

1.3 Одномерные иерархические базисные функции

Построение разреженной сетки основано на иерархическом базисе. В одномерном случае каждому узлу сопоставляется уровень ℓ и индекс i . Для граничного уровня $\ell = 0$ используются две базисные функции, отвечающие концам отрезка:

$$\varphi_{0,0}(x) = 1 - x, \quad \varphi_{0,1}(x) = x. \quad (5)$$

Для уровней $\ell > 0$ положение центра узла определяется как

$$c_{\ell,i} = \frac{i}{2^\ell}, \quad (6)$$

а радиус влияния равен

$$h_\ell = \frac{1}{2^\ell}. \quad (7)$$

Если аргумент находится вне области влияния узла, вклад базисной функции равен нулю.

Для линейного базиса используется выражение

$$\varphi_{\ell,i}^{(lin)}(x) = 1 - \frac{|x - c_{\ell,i}|}{h_\ell}, \quad |x - c_{\ell,i}| < h_\ell. \quad (8)$$

Для квадратичного базиса, который является основным в данной работе, применяется функция

$$\varphi_{\ell,i}^{(quad)}(x) = 1 - \left(\frac{|x - c_{\ell,i}|}{h_\ell} \right)^2, \quad |x - c_{\ell,i}| < h_\ell. \quad (9)$$

Квадратичный базис обеспечивает более гладкое поведение интерполянта внутри области влияния узла и лучше согласуется с аппроксимацией гладких отображений и операторов динамики, чем линейный вариант [3; 7].

1.4 Многомерный базис и тензорное произведение

Для точки $u = (u_1, \dots, u_d)$ вклад узла с уровнями $\ell = (\ell_1, \dots, \ell_d)$ и индексами $i = (i_1, \dots, i_d)$ задаётся произведением одномерных функций:

$$\Phi_{\ell,i}(u) = \prod_{k=1}^d \varphi_{\ell_k, i_k}(u_k). \quad (10)$$

Такое определение соответствует тензорному произведению одномерных базисов. Если хотя бы в одном измерении аргумент выходит из области влияния соответствующего узла, весь многомерный базис обращается в нуль. Именно это свойство обеспечивает локальность интерполяции и делает рекурсивный обход дерева вычислительно эффективным.

1.5 Иерархическое представление и коэффициенты избытка

Интерполянт на разреженной сетке строится в виде суммы иерархических вкладов:

$$\tilde{F}(u) = \sum_{(\ell,i) \in \mathcal{G}} \alpha_{\ell,i} \Phi_{\ell,i}(u), \quad (11)$$

где $\mathcal{G}$ — множество узлов сетки, а $\alpha_{\ell,i} \in \mathbb{R}^m$ — иерархические коэффициенты, или коэффициенты избытка [1].

Если обозначить через $u_{\ell,i}$ координаты центра узла, то коэффициент вычисляется как разность между точным значением и уже имеющейся интерполяцией:

$$\alpha_{\ell,i} = F(u_{\ell,i}) - \tilde{F}_{prev}(u_{\ell,i}). \quad (12)$$

Если норма коэффициента мала, это означает, что в данной точке текущее приближение уже хорошо описывает поведение функции. Наоборот, большие значения $\|\alpha_{\ell,i}\|$ указывают на необходимость дальнейшего уточнения.

1.6 Адаптивное уточнение и локальный критерий продолжения

В работе используется адаптивная стратегия построения сетки. Решение о создании дочерних узлов принимается по величине коэффициента избытка и по дополнительной проверке в специальных точках. Базовый критерий имеет вид

$$\|\alpha_{\ell,i}\| \leq \varepsilon, \quad (13)$$

где ε — допустимая погрешность.

Однако практика показывает, что опора только на значение коэффициента в центре узла может приводить к ложному завершению алгоритма. Этот эффект особенно

ярко проявляется на осциллирующих функциях.

1.7 Проблема вырожденного критерия на примере $\sin(x)$

Для адаптивных сеток существует характерная проблема. Рассмотрим функцию

$$f(x) = \sin(x) \quad (14)$$

на интервале $[0, 4\pi]$. На краях этого интервала и в его центре функция обращается в нуль:

$$f(0) = 0, \quad f(2\pi) = 0, \quad f(4\pi) = 0. \quad (15)$$

Если алгоритм анализирует только эти точки, он может сделать ложный вывод, что функция близка к прямой линии, то есть что дополнительное уточнение не требуется. Но фактически внутри интервала находятся выраженные экстремумы и осцилляции, которые не видны по такой редкой выборке.

Именно эта ситуация выявилась в ходе работы над дипломом. Для её устранения был реализован специальный механизм *якорных точек*, отсутствовавший в исходной статье научного руководителя [6]. Он позволяет вручную задать точки, в которых алгоритм обязан дополнительно контролировать ошибку и при необходимости продолжать уточнение сетки.

1.8 Механизм якорных точек как расширение базового метода

Пусть внутри области влияния текущего узла имеется контрольная точка $u^{(a)}$. Тогда дополнительно проверяется условие

$$\left\| F(u^{(a)}) - \tilde{F}(u^{(a)}) \right\| < \varepsilon. \quad (16)$$

Если оно нарушается, алгоритм обязан продолжать построение дочерних узлов, даже если центральный коэффициент избытка оказался малым.

Таким образом, якорные точки выполняют две функции:

- 1) предотвращают ложное «схлопывание» сетки на функциях с нулями и симметрией;
- 2) позволяют пользователю вручную указывать области, которые обязательно должны быть разрешены более подробно.

С математической точки зрения этот механизм можно интерпретировать как введение дополнительного множества ограничений на локальную ошибку. С практической точки зрения он является важным расширением базового алгоритма и одним из собственных результатов работы.

1.9 Порождение дочерних узлов

Пусть текущий узел имеет уровень $\ell_k > 0$ в k -м измерении. Тогда при уточнении по этой координате строятся два дочерних узла с уровнем

$$\ell'_k = \ell_k + 1, \quad (17)$$

и индексами

$$i_k^{left} = 2i_k - 1, \quad i_k^{right} = 2i_k + 1. \quad (18)$$

Все остальные уровни и индексы сохраняются. Такая формула отражает стандартное иерархическое ветвление.

В реализации дополнительно используется информация о направлении ошибки: если контрольная точка лежит слева от центра узла, приоритет отдаётся левой ветви, и наоборот. Это уменьшает число лишних вычислений.

1.10 Жёсткое ограничение числа узлов как механизм оптимизации

Помимо ограничения по точности и глубине уточнения, в работе реализована возможность жёсткого ограничения числа узлов. Если обозначить допустимый предел через N_{max} , то построение может быть принудительно остановлено при выполнении условия

$$|\mathcal{G}| \geq N_{max}. \quad (19)$$

Этот механизм важен с практической точки зрения. Он позволяет заранее ограничивать объём памяти и время построения сетки для сложных задач, когда пользователь предпочитает контролировать ресурсные затраты. Таким образом, ограничение числа узлов выступает дополнительным средством оптимизации наряду с выбором ε , типа базиса и максимального уровня.

1.11 Рекурсивное вычисление интерполянта

После построения сетки значение интерполянта в точке u вычисляется как сумма вкладов входных узлов и их потомков. Для узла n с коэффициентом α_n и базисной функцией Φ_n локальный вклад равен

$$V_n(u) = \alpha_n \Phi_n(u). \quad (20)$$

Если узел имеет потомков, добавляются их рекурсивные вклады:

$$S_n(u) = V_n(u) + \sum_{c \in \text{Children}(n)} S_c(u). \quad (21)$$

Локальность базисных функций обеспечивает быстрое завершение рекурсии: если $\Phi_n(u) = 0$, вклад всей соответствующей ветви равен нулю.

1.12 Связь с задачами моделирования динамических систем

Пусть динамическая система задаётся дифференциальным уравнением

$$\frac{dz}{dt} = f(z, t), \quad z(t_0) = z_0. \quad (22)$$

Тогда оператор перехода на временном шаге Δt можно записать как отображение

$$\Psi_{\Delta t}(z_0) = z(t_0 + \Delta t). \quad (23)$$

Подход, использованный в работе, позволяет интерполировать либо сам оператор перехода, либо его последовательные композиции. Это напрямую соотносится с тематикой моделирования динамических систем с интервальными параметрами, рассматриваемой в работе Морозова А. Ю. [6]. В дипломном проекте эта идея получила программное воплощение в модуле `compute` и была распространена на поддержку двух режимов работы со временем: через включение параметра t как дополнительной размерности и через итеративное применение одношагового оператора.

Выводы по главе

В первой главе рассмотрены математические основы метода, используемого в дипломной работе. Показано, что переход к разреженным сеткам позволяет уменьшить число узлов по сравнению с полной тензорной дискретизацией, а иерархическое представление на базе коэффициентов избытка обеспечивает естественный механизм адаптивного уточнения. Обосновано использование квадратичного базиса как более точного инструмента для аппроксимации гладких отображений. Отдельно описана проблема вырожденного критерия на функции $\sin(x)$ и предложенный в работе механизм якорных точек, позволяющий вручную принудить алгоритм уточнять сетку в значимых областях. Также показано, что аппарат разреженных сеток применим не только к обычным функциям, но и к операторам перехода, возникающим при численном решении дифференциальных уравнений. Эти положения образуют теоретическую базу для архитектуры и алгоритмов, реализованных в вычислительном модуле проекта.

2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И СТРУКТУР ДАННЫХ ПРОГРАММНОГО КОМПЛЕКСА

Во второй главе рассматривается архитектура разработанного программного комплекса. Основной акцент сделан на вычислительном модуле `compute`, поскольку именно он является центральным результатом дипломной работы. Серверный слой на gRPC и клиент визуализации на Flutter рассматриваются как вспомогательные компоненты, разработанные для удобства экспериментов, удалённого запуска и анализа результатов.

2.1 Роль вычислительного модуля в структуре проекта

Целью дипломной работы является реализация параллельных алгоритмов интерполяции на основе разреженных сеток с квадратичным базисом. Этой цели непосредственно соответствует модуль `compute`, в котором сосредоточены все ключевые математические и алгоритмические решения: представление узлов, вычисление базисных функций, построение адаптивной сетки, рекурсивная интерполяция, композиция операторов и поддержка режимов последовательного и параллельного построения.

Сервисный слой и пользовательский интерфейс не определяют математическое содержание работы, а лишь расширяют область практического применения созданного ядра. По этой причине архитектура программного комплекса должна рассматриваться прежде всего как архитектура вычислительного модуля, дополненная средствами визуализации и сетевого доступа.

2.2 Требования к вычислительному модулю

К модулю `compute` были предъявлены следующие требования:

- 1) поддержка многомерных входных данных и векторных выходных значений;
- 2) поддержка линейного и квадратичного базисов;
- 3) адаптивное уточнение сетки на основе коэффициентов избытка;
- 4) возможность учёта якорных точек для ручного управления уточнением;
- 5) поддержка последовательного и параллельного режимов построения;
- 6) возможность жёсткого ограничения числа узлов;
- 7) поддержка композиции отображений и последовательных итераций оператора;
- 8) совместимость с задачами прямой интерполяции и численного решения дифференциальных уравнений.

Именно выполнение этих требований определяет архитектурные решения, рассмотренные далее.

2.3 Центральный тип `AdaptiveSparseGrid`

Ключевой структурой вычислительного модуля является тип `AdaptiveSparseGrid`. Он объединяет в одном объекте:

- 1) границы области определения;
- 2) тип используемого базиса;
- 3) размерность входного и выходного пространств;
- 4) набор входных точек построения;
- 5) словарь всех узлов сетки.

Такое проектирование позволяет трактовать объект сетки одновременно как:

- 1) контейнер данных, описывающий уже построенный интерполянт;
- 2) исполняемый оператор, умеющий вычислять приближённое значение в новой точке;
- 3) основу для построения следующей итерации или композиции отображений.

Это архитектурное свойство особенно важно для задач моделирования динамических систем, где один и тот же объект должен не только хранить аппроксимацию, но и участвовать в последующих шагах вычисления.

2.4 Проектирование базовых структур данных

Для реализации метода были введены следующие типы.

Тип `Point`. Представляет собой вектор вещественных координат. Он используется как для входных точек, так и для значений функции и коэффициентов избытка. Использование единого векторного представления позволяет поддерживать как скалярные, так и векторные отображения без дублирования алгоритмов.

Тип `gridKey`. Определяет положение узла в иерархической структуре через массивы уровней и индексов. Такой ключ задаёт не только идентификатор узла, но и его геометрический смысл: уровень отвечает за масштаб, индекс — за положение центра.

Тип `node`. Содержит ключ, координаты центра в единичной области, коэффициент избытка и признак наличия потомков. Через эту структуру материализуется математическая модель адаптивной сетки.

Тип `entryPoint`. Описывает стартовую конфигурацию построения. Входные точки играют важную роль, поскольку именно от них запускаются независимые локальные ветви построения, что напрямую связано с возможностью распараллеливания.

В совокупности эти структуры образуют минимальное, но достаточное ядро вычислительной модели.

2.5 Архитектура построения сетки

С точки зрения архитектуры модуль `compute` организован вокруг процедуры построения, которая разделяется на два уровня.

На первом уровне формируются стартовые конфигурации уровней и индексов, соответствующие различным входным точкам. На втором уровне для каждой такой конфигурации запускается локальная задача адаптивного построения. Локальная задача:

- 1) создаёт начальный узел;
- 2) вычисляет коэффициент избытка;
- 3) проверяет критерии продолжения;
- 4) при необходимости порождает дочерние узлы;
- 5) рекурсивно или итеративно расширяет сетку до выполнения ограничений.

Это решение позволяет отделить глобальную организацию стартовых конфигураций от локальной логики уточнения. Такое разделение удобно и математически, и программно: оно делает код более модульным и упрощает последующую параллелизацию.

2.6 Архитектура поддержки `parallel` и `sequential build`

В модуле предусмотрены два режима работы: `BuildTypeSequential` и `BuildTypeParallel`. Их поддержка реализована не как два разных алгоритма, а как два режима исполнения одной и той же логики построения.

В последовательном режиме все локальные задачи выполняются по очереди в одном потоке. Этот режим удобен для проверки корректности, профилирования и сопоставления с параллельным вариантом.

В параллельном режиме независимые стартовые конфигурации обрабатываются одновременно. Архитектурное достоинство такого подхода состоит в том, что конкурентность не «вшивается» в математические операции над узлом, а накладывается поверх уже выделенных независимых задач. Благодаря этому ядро сохраняет логическую простоту, а параллелизация не разрушает его структуру.

2.7 Поддержка якорных точек как архитектурное расширение

В ходе разработки была выявлена проблема, когда адаптивный критерий, опирающийся только на коэффициент избытка в центре узла, может преждевременно остановить уточнение. Для решения этой проблемы в архитектуру вычислительного модуля был включён дополнительный механизм якорных точек.

С архитектурной точки зрения якорные точки представляют собой внешний набор контрольных точек, передаваемых в конструктор сетки. Они не изменяют базо-

вую формулу интерполяции, но влияют на принятие решения о продолжении построения. Это хороший пример расширения алгоритма без изменения его фундаментальной структуры: в ядро добавляется дополнительный слой условий, не нарушающий основной логики вычисления коэффициентов и рекурсивного обхода.

2.8 Жёсткое ограничение числа узлов как эксплуатационный параметр

Важным архитектурным решением является поддержка параметра `maxNodesInGrid`. Он задаёт верхнюю границу числа узлов, после которой построение должно завершиться принудительно. Наличие этого ограничения позволяет использовать вычислительный модуль не только в исследовательском режиме, но и в условиях ограниченных ресурсов.

С практической точки зрения это означает, что пользователь может выбрать один из двух режимов управления сложностью:

- 1) через точность и глубину уточнения;
- 2) через жёсткий лимит на размер сетки.

Такое решение особенно важно для серверного использования, где необходимо контролировать время ответа и объём данных, передаваемых клиенту.

2.9 Композиция отображений и метод следующей итерации

Одной из сильнейших сторон архитектуры модуля `compute` является поддержка композиции отображений через метод `MakeNextIteration`. В отличие от обычного вычислительного объекта, сетка в этом случае используется как промежуточный оператор: её результат подаётся на вход следующей функции, после чего строится новая аппроксимация.

С архитектурной точки зрения это превращает `AdaptiveSparseGrid` в универсальный вычислительный кирпич, пригодный для последовательного моделирования шагов эволюции. Именно поэтому модуль `compute` подходит не только для статической интерполяции, но и для задач динамики.

2.10 Поддержка дифференциальных уравнений на уровне архитектуры

Для работы с дифференциальными уравнениями используются два концептуально разных режима.

- 1) **Параметр t как интервальная неопределённость.** В этом случае время включается как дополнительная координата входного пространства. Тогда аппроксимируется отображение, зависящее как от начального состояния, так и от значения времени.

2) **Итеративный режим через `MakeNextIteration`.** В этом случае строится одношаговый оператор перехода, после чего он последовательно композиционно применяется несколько раз.

Архитектурная ценность этой двойной схемы состоит в том, что один и тот же вычислительный модуль способен поддерживать два разных численных сценария без изменения своего интерфейса. Это также делает возможным содержательное сравнение их производительности на уровне бенчмарков.

2.11 Вспомогательный серверный слой

Слой `internal/server` не является центральным результатом дипломной работы, но играет важную вспомогательную роль. Его задача — предоставить удобный удалённый интерфейс к вычислительному модулю. Сервер:

- 1) принимает параметры задачи;
- 2) интерпретирует пользовательские формулы;
- 3) при необходимости решает дифференциальное уравнение численным методом;
- 4) передаёт полученное отображение в `compute`;
- 5) сериализует построенную сетку в `protobuf`-ответ.

Таким образом, серверный слой следует рассматривать как адаптер между пользовательским запросом и вычислительным ядром, а не как самостоятельный предмет исследования.

2.12 Клиентская визуализация как вспомогательный инструмент

Аналогичную роль играет и Flutter-приложение. Оно предназначено для:

- 1) ввода параметров задачи;
- 2) запуска расчётов без работы из командной строки;
- 3) визуального анализа распределения узлов;
- 4) демонстрации результатов в наглядной форме.

Клиентская часть важна для практического применения проекта, однако архитектурный центр диплома находится именно в модуле `compute`. Flutter-интерфейс выступает как средство визуализации результатов работы этого модуля.

2.13 Наблюдаемость и эксплуатационные характеристики

Несмотря на вспомогательную роль серверного слоя, в проекте предусмотрены средства эксплуатационного мониторинга: метрики Prometheus, контроль длительности запроса, числа узлов и размера ответа. Эти механизмы полезны прежде всего для

оценки того, как параметры вычислительного модуля отражаются на внешних характеристиках системы.

Иначе говоря, наблюдаемость сервиса введена не ради самой серверной архитектуры, а как средство исследования поведения алгоритмов, реализованных в `compute`.

Выводы по главе

Во второй главе была рассмотрена архитектура программного комплекса с акцентом на вычислительный модуль `compute`, являющийся главным результатом дипломной работы. Показано, что именно в этом модуле сосредоточены все ключевые математические и алгоритмические решения: представление узлов, адаптивное уточнение, поддержка якорных точек, ограничение числа узлов, последовательный и параллельный режимы построения, а также композиция операторов и работа с дифференциальными уравнениями. Серверный слой на gRPC и клиент визуализации на Flutter рассмотрены как вспомогательные средства удалённого доступа и анализа результатов. Такая архитектура подчёркивает, что ядром проекта является именно реализация параллельных алгоритмов интерполяции на основе разреженных сеток.

3 РЕАЛИЗАЦИЯ ПАРАЛЛЕЛЬНЫХ АЛГОРИТМОВ, АНАЛИЗ РЕЗУЛЬТАТОВ И ПОДГОТОВКА ИЛЛЮСТРАЦИЙ

В третьей главе рассматривается практическая реализация вычислительного модуля и его интеграция с вспомогательными компонентами проекта. Основной акцент сделан на устройстве пакета `compute`, поскольку именно он реализует математический аппарат адаптивных разреженных сеток, поддержку линейного и квадратичного базисов, параллельное построение, контроль числа узлов и механизмы работы с динамическими системами.

3.1 Реализация вычислительного ядра на Go

Основная часть алгоритмов сосредоточена в пакете `compute`. Его центральным элементом является тип `AdaptiveSparseGrid`, инкапсулирующий параметры области, тип базиса, размерности входа и выхода, список входных точек и множество узлов. Выбор Go в качестве языка реализации вычислительного модуля обусловлен поддержкой конкурентных вычислений, удобной работой с функциями высшего порядка и простой интеграции в прикладной сервис [8].

Конструктор сетки выполняет валидацию входных данных, определяет размерность образа функции, вычисляет значения в контрольных точках и затем запускает построение. Важной особенностью является то, что вычислительное ядро принимает пользовательскую функцию как аргумент. Благодаря этому один и тот же код используется и для интерполяции аналитических отображений, и для операторов перехода, получаемых численным интегрированием.

В листинге 1 показан фрагмент конструктора, задающего общий сценарий создания сетки.

Листинг 1 — Создание адаптивной разреженной сетки

```
1 func NewAdaptiveSparseGridWithContext(  
2     ctx context.Context,  
3     funcEval func(Point) (Point, error),  
4     min, max Point,  
5     epsilon ScalarType,  
6     anchorPoints []Point,  
7     basisType BasisType,  
8     buildType BuildType,  
9     maxLevel int64,  
10    maxNodesInGrid int64,  
11 ) (*AdaptiveSparseGrid, error) {  
12     inDim := int64(len(min))  
13     if int64(len(max)) != inDim {  
14         return nil, fmt.Errorf("min and max must have the same dimension")
```

```

15     }
16
17     sampleOut, err := funcEval(min)
18     if err != nil {
19         return nil, err
20     }
21     outDim := int64(len(sampleOut))
22
23     grid := &AdaptiveSparseGrid{
24         min:      min,
25         max:      max,
26         basisType: basisType,
27         inDim:    inDim,
28         outDim:   outDim,
29         nodes:    make(map[string]node),
30     }
31
32     anchors, err := grid.calculateOriginalFunctionAtPoints(funcEval, anchorPoints)
33     if err != nil {
34         return nil, err
35     }
36
37     err = grid.build(ctx, funcEval, epsilon, anchors, buildType, maxLevel,
38         maxNodesInGrid)
39     if err != nil {
40         return nil, err
41     }
42     return grid, nil
43 }

```

В данном фрагменте видны основные свойства реализации: независимость от природы исходной функции, автоматическое определение размерности результата и наличие параметров, отвечающих за точность, тип базиса, режим построения и жёсткое ограничение числа узлов.

3.2 Ключи узлов и кодирование положения в сетке

Каждый узел сетки идентифицируется набором уровней и индексов. Для хранения узлов в отображении используется тип `gridKey`. Его строковое представление формируется бинарной упаковкой целочисленных массивов.

В листинге 2 показан соответствующий код.

Листинг 2 — Представление ключа узла сетки

```

1 type gridKey struct {
2     level []int64
3     index []int64
4 }

```



```

5
6 func (k gridKey) String() string {
7     totalLength := (len(k.level) + len(k.index)) * 8
8     result := make([]byte, totalLength)
9
10    pos := 0
11    for _, v := range k.level {
12        binary.LittleEndian.PutUint64(result[pos:], uint64(v))
13        pos += 8
14    }
15    for _, v := range k.index {
16        binary.LittleEndian.PutUint64(result[pos:], uint64(v))
17        pos += 8
18    }
19
20    return unsafe.String(&result[0], len(result))
21 }

```

Достоинство этого решения состоит в быстром поиске узла по его иерархическому положению. Это критически важно для построения потомков и для рекурсивной интерполяции по дереву узлов.

3.3 Реализация базисных функций

Математическая модель из первой главы непосредственно отражается в коде вычисления одномерного базиса. В зависимости от уровня, индекса и выбранного типа базиса вычисляется значение локальной функции влияния. Именно здесь программно различаются линейный и квадратичный варианты.

В листинге 3 приведена функция вычисления одномерного базиса.

Листинг 3 — Вычисление одномерной базисной функции

```

1 func eval1d(x float64, level int64, index int64, basisType BasisType) float64 {
2     if level == 0 {
3         if index == 0 {
4             return 1.0 - x
5         }
6         if index == 1 {
7             return x
8         }
9         return 0.0
10    }
11
12    h := 1.0 / float64(int64(1)<<level)
13    center := float64(index) * h
14    dist := math.Abs(x - center)
15
16    if dist >= h {
17        return 0.0

```

```

18     }
19
20     t := dist / h
21     switch basisType {
22     case BasisTypeLinear:
23         return 1.0 - t
24     case BasisTypeQuadratic:
25         return 1.0 - t*t
26     }
27     return 0.0
28 }

```

Данный фрагмент важен не только как реализация базиса, но и как точка прямого сопоставления двух режимов работы, сравнение которых затем проводится в бенчмарках.

3.4 Параллельный и последовательный режимы построения

Поддержка двух режимов сборки реализована без раздвоения математической логики. Независимые стартовые конфигурации либо вычисляются по очереди, либо запускаются как отдельные goroutine. На практике это позволяет сравнивать режимы `parallel` и `sequential` на одном и том же наборе задач.

В листинге 4 приведён фрагмент функции `build`, показывающий выбор между последовательным и параллельным режимами.

Листинг 4 — Параллельный запуск локальных задач построения

```

1  if buildType == BuildTypeParallel {
2      wg.Add(1)
3      go func(k gridKey) {
4          defer wg.Done()
5          res := g.buildGridTask(ctx, funcEval, epsilon, anchors, k, i, maxLevel,
6                                  maxNodesInGrid)
7          mu.Lock()
8          results = append(results, res)
9          mu.Unlock()
10     }(currentKey)
11 } else {
12     res := g.buildGridTask(ctx, funcEval, epsilon, anchors, currentKey, i, maxLevel,
13                             , maxNodesInGrid)
14     results = append(results, res)
15 }

```

Такой способ организации вычислений хорошо согласуется с логикой метода, поскольку параллелизуются именно независимые задачи первого уровня.

3.5 Создание узла и вычисление коэффициента избытка

При создании нового узла вычисляется значение исходной функции, затем значение текущей интерполяции в той же точке, после чего формируется коэффициент избытка как их разность.

В листинге 5 показан соответствующий фрагмент кода.

Листинг 5 — Формирование узла и вычисление коэффициента избытка

```
1 func (g *AdaptiveSparseGrid) createNode(  
2     funcEval func(Point) (Point, error),  
3     key gridKey,  
4     entryPoint *node,  
5     dimension int64,  
6     additionalNodes map[string]node,  
7 ) (*gridKey, error) {  
8     if _, exists := additionalNodes[key.String()]; exists {  
9         return nil, nil  
10    }  
11  
12    n := node{  
13        key:      key,  
14        centerUnit: make(Point, g.inDim),  
15    }  
16    for i := int64(0); i < g.inDim; i++ {  
17        n.centerUnit[i] = getCoord(key.level[i], key.index[i])  
18    }  
19  
20    realArg := g.toReal(n.centerUnit)  
21    etalon, err := funcEval(realArg)  
22    if err != nil {  
23        return nil, err  
24    }  
25  
26    interp := g.evaluateForDimAndEntryPoint(n.centerUnit, dimension, entryPoint,  
27        additionalNodes)  
28    etalon.Sub(interp)  
29    n.alpha = etalon  
30    additionalNodes[key.String()] = n  
31  
32    return &key, nil  
33 }
```

Этот код непосредственно реализует иерархическую природу метода: узел хранит не абсолютное значение функции, а именно добавочную информацию относительно уже построенного приближения.

3.6 Ключевой участок buildGrid: адаптация, якорные точки и ограничение числа узлов

Наиболее важным фрагментом реализации является внутренняя логика `buildGrid`, где одновременно решаются задачи анализа коэффициента избытка, учёта якорных точек, порождения дочерних узлов и контроля числа узлов. Именно этот участок лучше всего отражает реальное содержание вычислительного модуля, поэтому он является более показательным, чем элементарные операции над типом `Point`.

В листинге 6 приведён характерный фрагмент алгоритма.

Листинг 6 — Фрагмент алгоритма `buildGrid`: критерии продолжения и ограничение числа узлов

```
1 | for len(nodeQueue) > 0 {
2 |     currentKey := nodeQueue[0]
3 |     nodeQueue = nodeQueue[1:]
4 |
5 |     currentNode := newNodes[currentKey.String()]
6 |     activeEntryNode := newNodes[entryPointKeyString]
7 |
8 |     canContinue := false
9 |     directions := make([]direction, g.inDim)
10 |    for i := range directions {
11 |        directions[i] = directionBoth
12 |    }
13 |
14 |    if currentNode.alpha.Length() <= epsilon {
15 |        for i := range directions {
16 |            directions[i] = directionNone
17 |        }
18 |        canContinue = true
19 |    }
20 |
21 |    if canContinue {
22 |        for _, anchor := range anchors {
23 |            if currentNode.isPointInAffectZone(anchor.arg, g.inDim) {
24 |                evalRes := g.evaluateForDimAndEntryPoint(anchor.arg, dimension, &
25 |                    activeEntryNode, newNodes)
26 |                evalRes.Sub(anchor.ans)
27 |                if evalRes.Length() >= epsilon {
28 |                    canContinue = false
29 |                    affectDirs := getAffectDirection(anchor.arg, currentNode.key.
30 |                        level, currentNode.key.index, g.inDim)
31 |                    for i := int64(0); i < g.inDim; i++ {
32 |                        directions[i] |= affectDirs[i]
33 |                    }
34 |                }
35 |            }
36 |        }
37 |    }
38 | }
```

```

34     }
35 }
36
37 if canContinue {
38     continue
39 }
40
41 currentNode.hasChildren = true
42 newNodes[currentKey.String()] = currentNode
43
44 if maxNodesInGrid > 0 && int64(len(newNodes)) >= maxNodesInGrid {
45     break
46 }
47 }

```

Этот фрагмент отражает несколько существенных особенностей реализации:

- 1) базовое решение принимается по норме коэффициента избытка;
- 2) якорные точки могут отменить преждевременную остановку;
- 3) уточнение выполняется направленно;
- 4) рост сетки может быть жёстко ограничен параметром `maxNodesInGrid`.

Именно здесь проявляется одно из главных отличий реализованного модуля от базовой математической схемы, рассматриваемой в статье [6]: введение ручного механизма управления уточнением через якорные точки.

3.7 Рекурсивная интерполяция по построенной сетке

После завершения построения объект `AdaptiveSparseGrid` используется для быстрого вычисления приближённых значений. Интерполяция организована как рекурсивный обход дерева узлов: сначала учитывается вклад текущего узла, затем — вклад релевантных потомков.

В листинге 7 показан характерный фрагмент рекурсивного вычисления.

Листинг 7 — Рекурсивное вычисление интерполянта

```

1 func (g *AdaptiveSparseGrid) evaluateRecursive(
2     x Point,
3     n node,
4     processedNodes map[string]struct{},
5     additionalNodes map[string]node,
6 ) Point {
7     if _, exists := processedNodes[n.key.String()]; exists {
8         return make(Point, g.outDim)
9     }
10
11     processedNodes[n.key.String()] = struct{}{}
12     basis := evalBasis(x, n.key.level, n.key.index, g.basisType, g.inDim)
13

```

```

14     if math.Abs(basis) < 1e-9 {
15         return make(Point, g.outDim)
16     }
17
18     value := n.alpha.Clone()
19     value.MulScalar(basis)
20
21     if n.hasChildren {
22         for i := int64(0); i < g.inDim; i++ {
23             if n.key.level[i] != 0 {
24                 child, exists := g.getChildForDimAndArg(n, x, i, additionalNodes)
25                 if exists {
26                     value.Add(g.evaluateRecursive(x, child, processedNodes,
27                                             additionalNodes))
28                 }
29             }
30         }
31     }
32     return value
33 }

```

Программный смысл этого кода совпадает с теоретической формулой суммы иерархических вкладов. Если значение базиса практически нулевое, соответствующая ветвь обхода мгновенно отсекается.

3.8 Два режима работы с дифференциальными уравнениями

Для задач динамики в проекте поддерживаются два принципиально разных подхода.

Первый подход: параметр t как интервальная неопределённость. В этом случае время добавляется как ещё одна координата пространства аргументов. Тогда строится сетка для отображения

$$(x_0, t) \mapsto x(t). \quad (24)$$

Преимущество этого подхода состоит в том, что одна построенная сетка сразу описывает поведение решения на целом временном интервале. Недостаток — рост размерности задачи и, как следствие, рост вычислительной сложности.

Второй подход: итеративное построение через `MakeNextIteration`. В этом случае сначала строится одношаговый оператор перехода на интервале длины 1, а затем он применяется последовательно несколько раз. Преимущество такого подхода состоит в том, что размерность пространства не увеличивается, но приходится выполнять серию последовательных построений.

Именно сопоставление этих двух подходов является одним из ключевых содержа-

тельных результатов экспериментальной части, поскольку оно показывает, что разные трактовки времени приводят к разным профилям вычислительной сложности.

3.9 Вспомогательный серверный слой

Пакет `internal/server` реализует удалённый интерфейс к вычислительному ядру. Объект `Formula` использует библиотеку `expr` для интерпретации пользовательских выражений [9]. При работе с дифференциальными уравнениями сервер численно интегрирует систему методом Рунге–Кутты–Фельберга [4], после чего передаёт в модуль `compute` уже готовое отображение.

С точки зрения архитектуры серверный слой является адаптером, а не предметом основной новизны работы. Его значение состоит в том, что он позволяет использовать вычислительный модуль удалённо и удобно подключать визуализацию.

3.10 Клиентская визуализация

Клиентская часть на Flutter реализует ввод параметров, запуск вычислений и отображение узлов сетки [10]. Она важна как средство анализа результатов, но не является математическим ядром работы. Основной результат диплома по-прежнему сосредоточен в модуле `compute`; клиент выступает как инструмент демонстрации и визуального исследования структуры построенного интерполянта.

3.11 Анализ структуры бенчмарков

Для оценки эффективности реализации был расширен набор бенчмарков. В текущей версии проекта измеряются:

- 1) зависимость времени построения от размерности задачи и размера области;
- 2) сравнение последовательного и параллельного режимов построения;
- 3) сравнение линейного и квадратичного базисов при различных значениях ε ;
- 4) влияние якорных точек на сложность построения;
- 5) сравнение двух подходов к работе с дифференциальными уравнениями;
- 6) влияние жёсткого ограничения числа узлов;
- 7) стоимость вычисления значения по уже построенной сетке.

Такой набор экспериментов нельзя считать тривиальным, поскольку он отражает не только абсолютное время работы, но и влияние архитектурных решений модуля `compute` на разные режимы использования.

3.12 Анализ полученных данных

По содержанию бенчмарков можно выделить несколько наиболее важных групп выводов.

Во-первых, линейный и квадратичный базисы должны сравниваться не только по качеству аппроксимации, но и по времени построения и стоимости последующего вычисления. Квадратичный базис обычно оказывается более затратным на локальном уровне, но при этом позволяет получать более гладкое приближение и потенциально уменьшать необходимую глубину уточнения. Поэтому содержательный анализ должен учитывать обе стороны — локальную стоимость и глобальную эффективность.

Во-вторых, сравнение `parallel` и `sequential` показывает практическую ценность распараллеливания стартовых конфигураций. Это сравнение особенно важно для темы диплома, поскольку напрямую связано с заявленной реализацией параллельных алгоритмов.

В-третьих, сравнение двух режимов работы с дифференциальными уравнениями демонстрирует, что параметр времени может быть учтён либо как отдельная координата, либо через последовательную композицию одношагового оператора. Эти подходы различаются по архитектурному смыслу и по производительности, а потому требуют отдельного разбора в итоговой аналитической главе.

В-четвёртых, механизм жёсткого ограничения числа узлов оказывается важным компромиссом между точностью и затратами. Он даёт возможность использовать метод в ресурсно ограниченной среде и должен рассматриваться как существенный элемент практической оптимизации.

Выводы по главе

В третьей главе была рассмотрена практическая реализация вычислительного модуля и его вспомогательной инфраструктуры. Показано, каким образом математический аппарат разреженных сеток отражён в коде пакета `compute`, как реализованы базисные функции, коэффициенты избытка, адаптивное уточнение, якорные точки, ограничение числа узлов, рекурсивная интерполяция и два режима работы с дифференциальными уравнениями. Отдельно рассмотрены архитектурная роль серверного слоя и клиентской визуализации, а также структура расширенного набора бенчмарков. Всё это подтверждает, что основное содержание дипломной работы сосредоточено именно в вычислительном модуле, а дополнительные компоненты служат средствами его практического применения и анализа.

4 ОБОБЩЕНИЕ РЕЗУЛЬТАТОВ И НАПРАВЛЕНИЯ ДАЛЬНЕЙШЕГО РАЗВИТИЯ

В четвёртой главе проводится детальный анализ экспериментальных результатов и архитектурных решений, реализованных в вычислительном модуле `compute`. Основной объём главы посвящён интерпретации бенчмарков, поскольку именно они позволяют количественно оценить влияние выбора базиса, режима построения, размерности задачи, способа работы с дифференциальными уравнениями, якорных точек и ограничения числа узлов на производительность алгоритма. Дополнительно в данной главе приводится сравнение Go- и C++-реализаций библиотеки, выполненных на одинаковом наборе тестов.

4.1 Соответствие результатов поставленным задачам

Поставленные в дипломной работе задачи были связаны не только с реализацией математического метода, но и с исследованием его поведения в различных режимах. По итогам работы можно утверждать, что:

- 1) реализован вычислительный модуль для адаптивной интерполяции на разреженных сетках;
- 2) поддержаны линейный и квадратичный базисы;
- 3) реализованы последовательный и параллельный режимы построения;
- 4) введён механизм якорных точек для устранения ложной стабилизации алгоритма;
- 5) добавлен параметр жёсткого ограничения числа узлов;
- 6) реализованы два способа работы с дифференциальными уравнениями;
- 7) подготовлен расширенный набор бенчмарков, позволяющий исследовать поведение алгоритма в нетривиальных сценариях;
- 8) выполнено сопоставление результатов Go- и C++-версий библиотеки.

Следовательно, дипломная работа завершилась не только созданием реализации, но и построением полноценной экспериментальной базы для анализа разработанного метода.

4.2 Общая характеристика экспериментальной программы

Измерения для Go-версии выполнялись на платформе Apple M4 Pro под управлением macOS, архитектура — arm64. Все результаты получены командой `go test -bench . ./compute`. Для анализа использовались времена выполнения, объём выделяемой памяти и число аллокаций.

Для C++-версии библиотеки использовался тот же набор сценариев, что и ранее, поэтому межъязыковое сравнение сохраняет актуальность. В C++ для параллельного исполнения применялся механизм `std::async` с флагом `std::launch::async`. Это позволяет сопоставлять не только алгоритмические режимы, но и особенности двух разных моделей исполнения.

В экспериментальную программу вошли следующие группы тестов:

- 1) зависимость времени построения от размерности задачи и размера области;
- 2) сравнение последовательного и параллельного режимов построения;
- 3) сравнение линейного и квадратичного базисов при разных значениях точности;
- 4) исследование влияния якорных точек на построение сетки;
- 5) сравнение двух способов работы с дифференциальными уравнениями;
- 6) анализ влияния жёсткого ограничения числа узлов;
- 7) измерение стоимости вычисления значения по уже построенной сетке;
- 8) сравнение построения 4-мерной функции на декартовом произведении всех режимов `BuildType` и `BasisType`.

Во всех экспериментах использовались явно заданные области определения, параметры точности и тестовые функции. Это важно, поскольку стоимость построения сетки определяется не только размерностью, но и конкретной структурой функции, распределением осцилляций по области и выбранным значением ε .

4.3 Зависимость времени построения от размерности и размера области в Go-реализации

Бенчмарк `BenchmarkDomainScalingDifferentDimensions` исследует функцию вида

$$f(x_1, \dots, x_d) = \sum_{k=1}^d \left(\sin(kx_k) \cos\left(\frac{x_1 x_k}{k+1}\right) + \tan(0.1x_k) \right). \quad (25)$$

В отличие от более простого варианта, здесь при $k > 1$ появляется зависимость от произведения $x_1 x_k$, поэтому задача становится более показательной для оценки поведения сетки при смешанной зависимости координат.

Измерения выполнялись на областях

$$\Omega_{d,s} = [0, s\pi]^d, \quad s \in \{1, 2, 4\}, \quad d \in \{1, 2, 3\}, \quad (26)$$

при фиксированной точности

$$\varepsilon = 0.001, \quad (27)$$

их результаты продемонстрированы на рисунке 1

Для одномерной задачи при масштабе π время составило 13354 нс/оп, для масштаба 2π — 36450 нс/оп, а для 4π — 104227 нс/оп. Для двумерной задачи получены 80327, 149063 и 336487 нс/оп соответственно. Для трёхмерной задачи время возрастает до 636276 нс/оп при масштабе π , 766213 нс/оп при 2π и 1638044 нс/оп при 4π .

В этой постановке зависимость от размера области становится монотонной: увеличение масштаба приводит к росту времени построения во всех размерностях. Это ожидаемо, поскольку на большем интервале функция содержит больше выраженных изменений, а смешанные множители усиливают необходимость строить не только одномерные ветви, но и внутренние узлы сетки.

Переход от одной размерности к двум и трём также заметно увеличивает стоимость. Например, при масштабе 4π время возрастает от 104.2 мкс в 1D до 336.5 мкс в 2D и до 1.64 мс в 3D. Следовательно, эксперимент подтверждает, что разреженная сетка смягчает рост сложности, но при наличии смешанных зависимостей размерность и масштаб области остаются ключевыми факторами затрат.

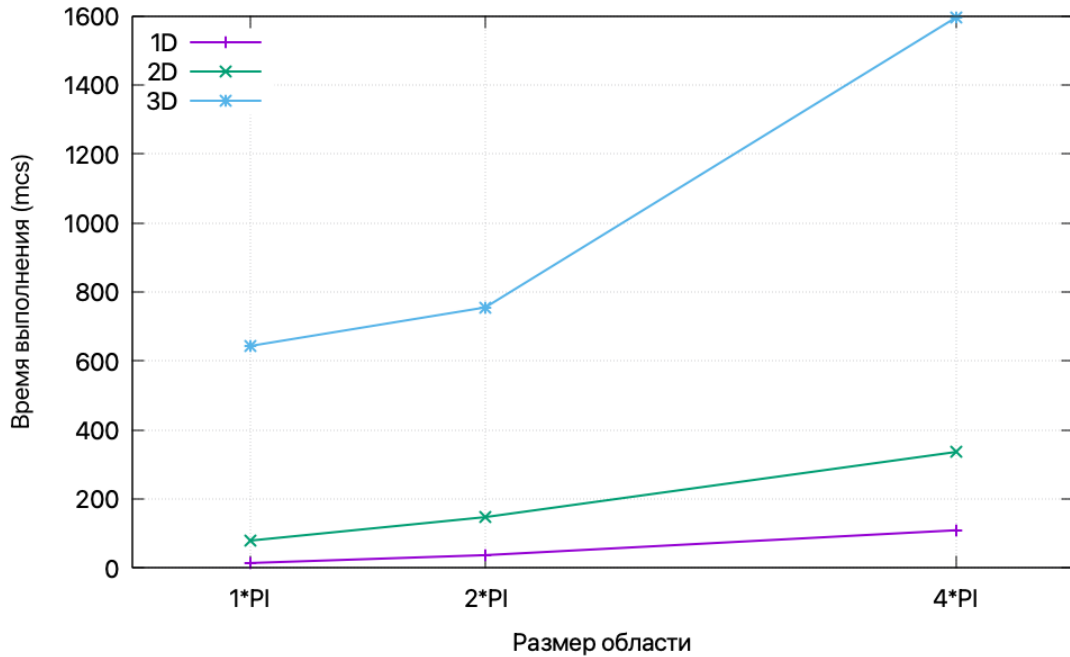


Рисунок 1 — Зависимость времени построения сетки от размерности задачи и размера области

4.4 Сравнение последовательного и параллельного построения в Go-реализации

Бенчмарк `BenchmarkParallelVsSequentialBuild` рассматривает двумерную функцию

$$f(x, y) = \sin(3x) + 0.35 \cos(25y) + 0.2 \sin(7x + 2.5) - 0.15 \cos(11y - 1.0) \quad (28)$$

на области

$$\Omega = [0, 2\pi] \times [0, 2\pi] \quad (29)$$

при

$$\varepsilon = 0.001. \quad (30)$$

Для линейного базиса последовательное построение дало 7377917 нс/оп, а параллельное — 5495296 нс/оп. Для квадратичного базиса значения составили 2867911 и 2324588 нс/оп соответственно. Таким образом, на данной двумерной функции параллельный режим даёт выигрыш для обоих базисов.

Такой результат показывает, что для нетривиальных двумерных задач объём локальной работы в независимых ветвях дерева уже достаточен для того, чтобы компенсировать накладные расходы на конкурентное исполнение. Особенно заметно это для линейного базиса, где абсолютный выигрыш по времени оказывается наиболее выраженным.

С инженерной точки зрения это важное наблюдение. Оно показывает, что оценивать параллельный режим нужно не на искусственно простых конфигурациях, а на задачах, где структура сетки действительно нетривиальна. Именно в таком режиме параллельная архитектура вычислительного модуля проявляет свою практическую ценность.

4.5 Сравнение линейного и квадратичного базисов в Go-реализации

Одним из центральных вопросов дипломной работы является сравнение линейного и квадратичного базисов. В данном бенчмарке рассматривается функция

$$f(x, y) = \sin(15x) + 0.6 \cos(40y) + 0.2 \sin(31x + 0.3) - 0.12 \cos(19y - 0.7) + \sin(xy) \quad (31)$$

на области

$$\Omega = [0, \pi] \times [0, \pi], \quad (32)$$

а точность принимает значения

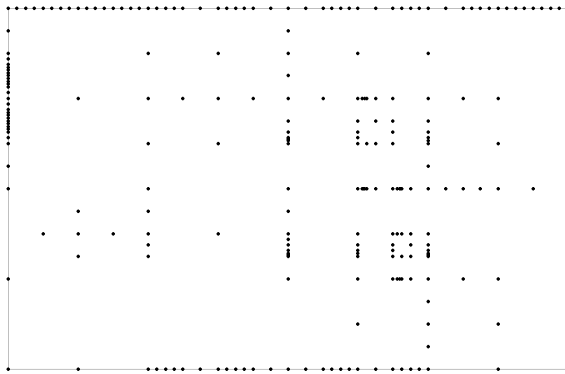
$$\varepsilon \in \{0.01, 0.005, 0.001, 10^{-4}, 10^{-5}, 5 \cdot 10^{-6}, 10^{-6}\}. \quad (33)$$

Результаты Go-реализации показывают устойчивое преимущество квадратичного базиса. Для $\varepsilon = 0.01$ линейный базис дал 2307763 нс/оп, а квадратичный — 978210 нс/оп. Для $\varepsilon = 0.005$ значения составили 2955778 и 1165347 нс/оп соответственно. Для $\varepsilon = 0.001$ получены 7544096 и 2938643 нс/оп.

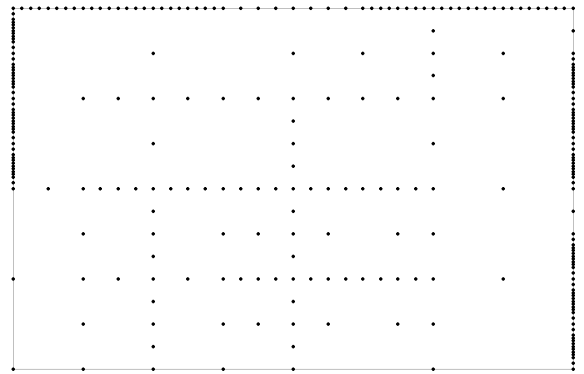
Эти данные показывают, что квадратичный базис оказывается выгоднее не только теоретически, но и практически. Более выразительная локальная форма квадратичного базиса позволяет достигать требуемой точности на меньшем числе уточнений. В

результате усложнение формулы отдельной базисной функции компенсируется уменьшением общего размера дерева построения.

Особенно наглядно это проявляется при $\varepsilon = 0.001$: линейный базис требует примерно 7.54 мс на построение, тогда как квадратичный — около 2.94 мс. Следовательно, по мере ужесточения требования к точности преимущество квадратичного базиса не ослабевает, а усиливается. Это означает, что более качественный базис здесь оказывается одновременно и более быстрым. На рисунках 2 и 3 продемонстрированы изображения полученных сеток при различных параметрах, а на рисунке 4 - результаты бэнчмарка.

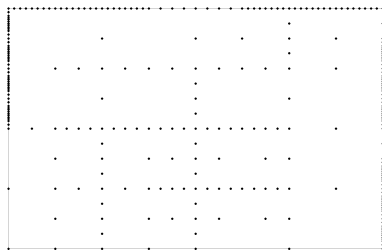


Линейный базис, $\varepsilon = 0.01$

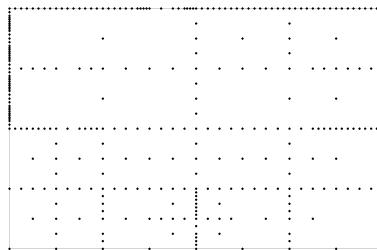


Квадратичный базис, $\varepsilon = 0.01$

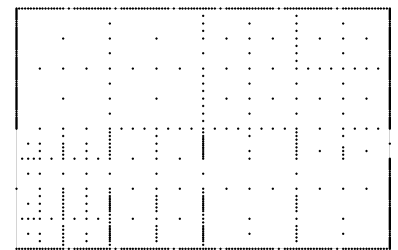
Рисунок 2 — Сравнение расположения узлов для линейного и квадратичного базисов



$\varepsilon = 0.01$



$\varepsilon = 0.005$



$\varepsilon = 0.001$

Рисунок 3 — Изменение плотности узлов квадратичной сетки при уменьшении ε

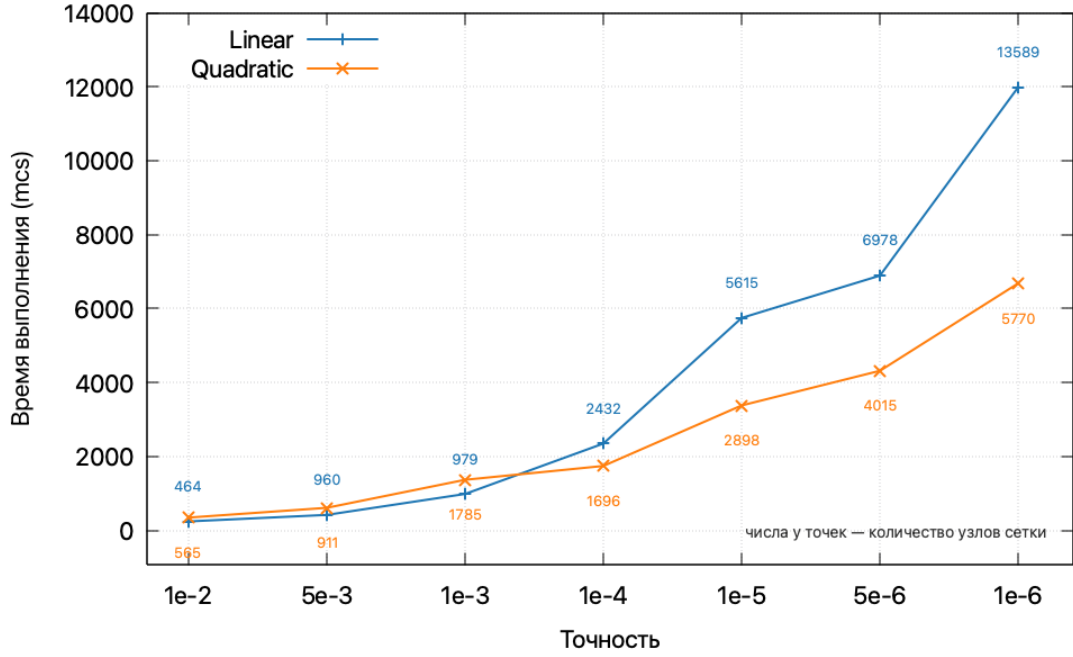


Рисунок 4 — Сравнение времени построения сетки для линейного и квадратичного базисов при различных значениях ϵ

4.6 Влияние якорных точек на производительность и корректность построения

Бенчмарк `BenchmarkAnchorPointsImpact` был специально включён в экспериментальную программу для исследования механизма якорных точек. Рассматривалась функция

$$f(x) = \sin(x) \quad (34)$$

на интервале

$$\Omega = [0, 4\pi] \quad (35)$$

при

$$\epsilon = 0.001. \quad (36)$$

В качестве дополнительных контрольных точек использовались значения

$$x^{(1)} = \pi, \quad x^{(2)} = 3\pi. \quad (37)$$

Построение без якорных точек заняло 1871 нс/оп, а с якорными точками — 2195 нс/оп. Увеличение времени здесь не является недостатком алгоритма. Напротив, оно отражает содержательный факт: алгоритм перестаёт преждевременно завершать построение и начинает учитывать дополнительные области, которые в базовом режиме были бы ошибочно проигнорированы.

Таким образом, эксперимент подтверждает практическую значимость введённого механизма. Якорные точки действительно влияют на поведение алгоритма и позво-

ляют принудительно направлять построение в важные зоны пространства аргументов. Для осциллирующих функций это особенно важно, поскольку базовый критерий по коэффициенту избытка в центре узла может оказаться недостаточным.

4.7 Два подхода к работе с дифференциальными уравнениями в Go-реализации

Бенчмарк `BenchmarkDifferentialEquationApproaches` сравнивает два способа построения сетки для двумерной динамической системы

$$\frac{dx}{dt} = -\frac{y}{1 + \sqrt{x^2 + y^2}}, \quad \frac{dy}{dt} = -\frac{x}{1 + \sqrt{x^2 + y^2}}. \quad (38)$$

Этот пример важен тем, что он соответствует двумерному оператору перехода, а не одномерному скалярному затуханию. Поэтому сравнение двух подходов становится более показательным с точки зрения структуры итоговой сетки.

В первом подходе время рассматривается как дополнительная координата, а оператор строится на области

$$(x_0, y_0, t) \in [-1, 1] \times [0, 1] \times [0, t_{max}]. \quad (39)$$

Во втором подходе строится одношаговый оператор на области

$$(x_0, y_0) \in [-1, 1] \times [0, 1] \quad (40)$$

с последующим итеративным применением `MakeNextIteration`. Во всех сериях использовалась точность

$$\varepsilon = 0.001, \quad t_{max} \in \{2, 5, 10, 20\}. \quad (41)$$

Полученные результаты показывают, что итеративный подход оказывается быстрее на всём рассмотренном диапазоне горизонтов времени. При $t_{max} = 2$ построение с дополнительной координатой времени заняло 3381 мкс/оп, тогда как итеративный подход — 1750 мкс/оп. При $t_{max} = 5$ значения составили 32711 и 15275 мкс/оп соответственно. При $t_{max} = 10$ получено 266993 мкс/оп для режима с дополнительной координатой и 121045 мкс/оп для итеративного. При $t_{max} = 20$ разрыв сохраняется: 1811110 против 1068985 мкс/оп.

Таким образом, итеративное построение оказывается выгоднее по времени на всех горизонтах: оно работает в пространстве меньшей размерности и строит сетку с меньшим числом узлов. Это подтверждается и числом узлов: при $t_{max} = 20$ итеративному подходу потребовалось около 207 тысяч узлов суммарно по всем срезам, тогда как режиму с дополнительной координатой — около 96 тысяч узлов в едином объёме. Несмотря на меньшее число узлов, подход с дополнительной координатой обходится дороже, поскольку каждый его узел соответствует более длинному пути численного

интегрирования по времени. При этом важно отметить ограничение итеративного подхода: он даёт значения только в целочисленных узлах модельного времени, поэтому для произвольного дробного момента t всё равно требуется режим с параметром t как дополнительной координатой. На рисунке 5 продемонстрирован результат бэнчмарка, а на рисунке 6 - изображения сеток, полученных обоими методами.

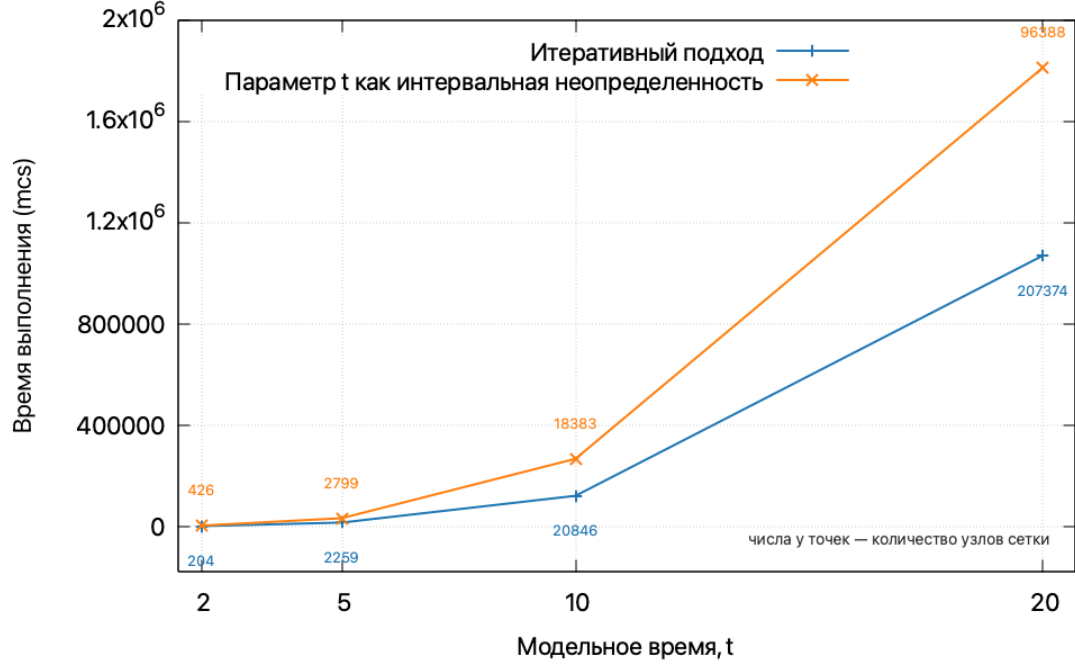


Рисунок 5 — Сравнение двух способов работы с двумерной динамической системой: t как дополнительная размерность и итеративный режим через `MakeNextIteration`

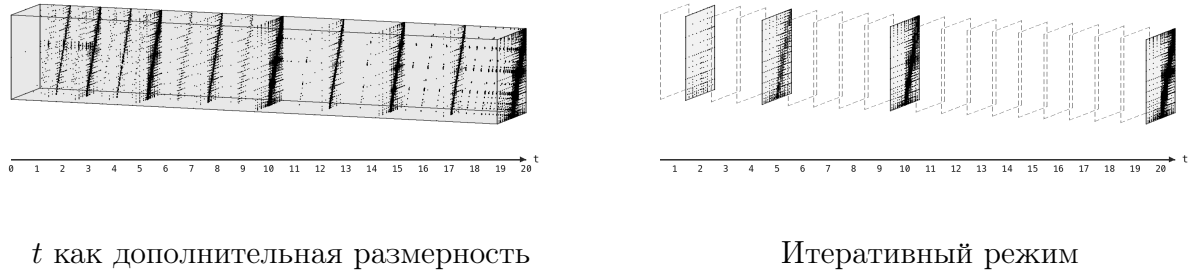


Рисунок 6 — Сравнение расположения узлов для двух подходов к двумерной динамической системе при $t_{max} = 5$ и $\varepsilon = 0.001$

4.8 Жёсткое ограничение числа узлов как средство оптимизации

Бэнчмарк `BenchmarkNodeLimitOptimization` исследует функцию

$$f(x, y) = \sin(5x) + \cos(33y) + 0.22 \sin(17x + 0.4) - 0.18 \cos(21y - 0.2) \quad (42)$$

на области

$$\Omega = [0, 2\pi] \times [0, 2\pi] \quad (43)$$

при

$$\varepsilon = 0.001, \quad (44)$$

а ограничение числа узлов принимает значения

$$N_{max} \in \{0, 200, 500, 1000\}. \quad (45)$$

Здесь значение $N_{max} = 0$ означает отсутствие жёсткого ограничения.

Без ограничения время составляет 4833921 нс/оп. При лимите 200 узлов время снижается до 972821 нс/оп. При лимите 500 получаем 2160798 нс/оп, а при лимите 1000 — 3104964 нс/оп. Следовательно, параметр `maxNodesInGrid` действительно работает как сильный рычаг управления затратами даже на более содержательной двумерной функции.

Этот эксперимент особенно важен для практического использования. Он показывает, что ограничение числа узлов следует рассматривать как полноценный механизм оптимизации, а не как второстепенную техническую настройку. Уменьшение лимита радикально сокращает время построения, хотя и может приводить к ухудшению точности интерполяции. Именно в этом и заключается его практический смысл: исследователь получает возможность явно управлять компромиссом между качеством и затратами.

4.9 Стоимость вычисления значения после построения

В прикладных задачах сетка строится не ради самого построения, а ради многократного последующего использования. Поэтому отдельный интерес представляет стоимость вызова `Evaluate()` после того, как сетка уже готова. Именно эту сторону отражает бенчмарк `BenchmarkEvaluationCostAfterBuild`, использующий ту же функцию, что и сравнение базисов:

$$f(x, y) = \sin(15x) + 0.6 \cos(40y) + 0.2 \sin(31x + 0.3) - 0.12 \cos(19y - 0.7) + \sin(xy) \quad (46)$$

на области

$$\Omega = [0, \pi] \times [0, \pi] \quad (47)$$

при

$$\varepsilon = 0.001, \quad (48)$$

а оценивание выполняется в точке

$$(x, y) = (0.73, 1.11). \quad (49)$$

Результаты показывают, что квадратичный базис и здесь даёт заметный выигрыш. В Go для линейной сетки получено 43343992 нс/оп, а для квадратичной — 37695183 нс/оп. Это означает, что квадратичная сетка остаётся быстрее и на этапе вычисления значения, хотя разница уже не столь радикальна, как на этапе построения. В

C++ наблюдается аналогичная качественная картина: 21612112 нс для линейной сетки и 19527708 нс для квадратичной.

Следовательно, преимущество квадратичного базиса проявляется не только при построении, но и при последующей эксплуатации сетки. Это особенно важно с прикладной точки зрения: если интерполянт строится один раз, а затем используется многократно, стоимость вызова `Evaluate()` становится критически важной характеристикой.

4.10 Четырёхмерная функция: сравнение всех сочетаний BuildType и BasisType

Дополнительный интерес представляет бенчмарк построения 4-мерной функции на декартовом произведении всех сочетаний BuildType и BasisType. Рассматривается функция

$$f(x_1, x_2, x_3, x_4) = \sin(30x_1) \cos(22.2323x_2) + \sin(70x_3) - 0.15 \sin(x_4) + \cos(x_1x_3) \quad (50)$$

на области

$$\Omega = [0, \pi]^4 \quad (51)$$

при значительно более строгом параметре точности

$$\varepsilon = 1 \cdot 10^{-7}. \quad (52)$$

В Go получены следующие результаты: 25193296833 нс/оп для `sequential_linear`, 14266548209 нс/оп для `sequential_quadratic`, 5741631375 нс/оп для `parallel_linear` и 3369355417 нс/оп для `parallel_quadratic`, которые продемонстрированы на рисунке 7. Из них видно, что квадратичный базис оказывается существенно быстрее линейного и в последовательном, и в параллельном режиме. Разница здесь уже имеет многократный характер.

В C++-реализации для того же сценария получены существенно большие абсолютные времена: 13491483225 нс для `sequential_linear`, 7021932158 нс для `sequential_quadratic`, 4406079104 нс для `parallel_linear` и 2235663521 нс для `parallel_quadratic`. В отличие от малых двумерных тестов, здесь параллельный режим в C++ даёт принципиальный выигрыш, сокращая время построения примерно в 2.9–3.3 раза.

Кроме того, на 4-мерной функции параллельный режим улучшает результат для линейного базиса по сравнению с последовательным, тогда как для квадратичного базиса разница между режимами оказывается очень малой, и последовательный вариант немного выигрывает. Это означает, что при хорошем качестве аппроксимации квадратичного базиса основное ускорение достигается за счёт уменьшения числа уточнений, а не обязательно за счёт конкурентного исполнения.

В обеих реализациях квадратичный базис остаётся быстрее линейного, однако

влияние BuildType различается количественно. В Go основное ускорение даёт выбор квадратичного базиса, а параллельный режим особенно помогает линейному варианту. В C++ тяжёлая 4-мерная постановка, напротив, делает параллельный режим ключевым фактором ускорения. Тем самым 4-мерный бенчмарк показывает, что относительная роль базиса и параллелизма зависит от размерности и от конкретной реализации.

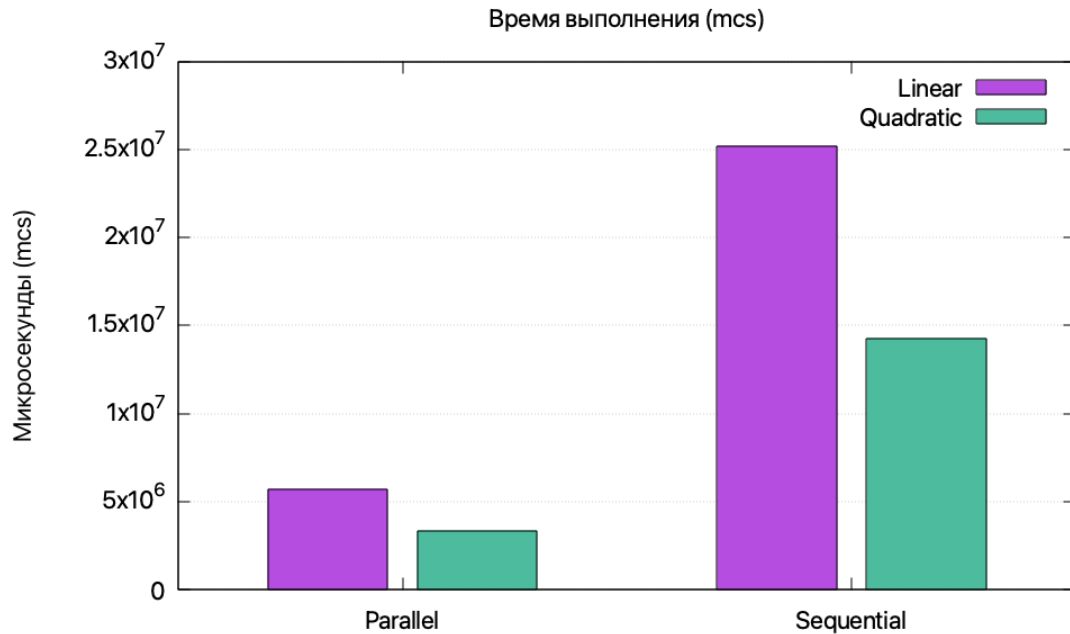


Рисунок 7 — Сравнение всех сочетаний BuildType и BasisType на 4-мерной функции

4.11 Сравнение Go- и C++-реализаций библиотеки

Сопоставление двух реализаций усиливает инженерную ценность работы и позволяет говорить не только о корректности алгоритма, но и о свойствах различных способов его программного воплощения.

4.11.1 Сравнение последовательных реализаций

В последовательном режиме C++-реализация в большинстве сценариев показывает более низкое время построения, чем Go-реализация. Например, для бенчмарка масштабирования по размерности и области в 1D при масштабе π получено 9238 нс против 13354 нс в Go. В 2D при масштабе π — 55230 нс против 80327 нс, а в 3D при масштабе π — 424831 нс против 636276 нс. При масштабе 4π сохраняется та же картина: 67249 нс против 104227 нс в 1D, 234580 нс против 336487 нс в 2D и 1013889 нс против 1638044 нс в 3D. Следовательно, на данном сценарии C++ быстрее Go, но обе реализации демонстрируют одинаковую качественную зависимость: рост размерности и расширение области увеличивают стоимость построения.

Такой результат ожидаем. C++-версия использует более низкоуровневую модель исполнения и, вероятно, эффективнее работает с временными объектами и размещением данных в памяти. Поэтому в чисто последовательном режиме C++ закономерно

оказывается быстрее.

4.11.2 Сравнение параллельных реализаций

Наиболее интересная часть сопоставления связана с параллельным режимом. В бенчмарке `ParallelVsSequentialBuild` C++ показывает близкие результаты для всех четырёх комбинаций: 172579 нс для `SEQUENTIAL_LINEAR`, 170977 нс для `SEQUENTIAL_QUADRATIC`, 173430 нс для `PARALLEL_LINEAR` и 160735 нс для `PARALLEL_QUADRATIC`. Для Go на той же группе сценариев получены 7377917 и 2867911 нс для последовательных режимов, а также 5495296 и 2324588 нс для параллельных.

Следовательно, сравнение остаётся содержательным: в Go параллельный режим действительно даёт ускорение на нетривиальной двумерной функции, тогда как в C++ выигрыш выражен только для квадратичного базиса и остаётся умеренным. Это подчёркивает, что эффективность распараллеливания определяется не только алгоритмом, но и стоимостью запуска задач и организацией памяти в конкретной реализации.

При этом качественное замечание о модели исполнения сохраняется. В Go конкурентный код поддерживается встроенным scheduler, который эффективно управляет большим числом лёгких goroutine. В C++ использовался `std::async` с флагом `async`. Поэтому важно подчеркнуть не абсолютное превосходство одной платформы над другой, а различие в моделях конкурентного исполнения и в том, как они ведут себя на задачах разного масштаба.

4.11.3 Сравнение Go- и C++-реализаций на задаче Коши

Для прямого сопоставления двух реализаций был использован итеративный подход к построению сетки для двумерной динамической системы при тех же значениях $t_{max} \in \{2, 5, 10, 20\}$ и точности $\varepsilon = 0.001$. Этот сценарий удобен тем, что охватывает весь конвейер вычислений: численное интегрирование оператора перехода, построение разреженной сетки и последовательную композицию одношаговых операторов.

Результаты, представленные на рисунке 8, показывают устойчивое преимущество C++-реализации на всём диапазоне модельного времени. При $t_{max} = 2$ C++-версия построила сетку за 988 мкс против 1750 мкс у Go. При $t_{max} = 5$ значения составили 8357 и 15275 мкс соответственно, при $t_{max} = 10$ — 46050 и 121045 мкс, а при $t_{max} = 20$ — 402073 против 1068985 мкс.

Существенно, что с ростом горизонта времени разрыв между реализациями увеличивается: если при $t_{max} = 2$ Go медленнее примерно в 1.8 раза, то при $t_{max} = 20$ — уже почти в 2.7 раза. Это объясняется тем, что на длинных горизонтах доминирует стоимость самих вычислений — численного интегрирования и обхода сетки, — а не накладные расходы на запуск параллельных задач. В этом режиме более низкоуровневая модель исполнения C++ и эффективная работа с временными объектами и размещением данных в памяти дают C++-версии решающее преимущество, несмотря на менее гибкую модель конкурентного исполнения через `std::async`.

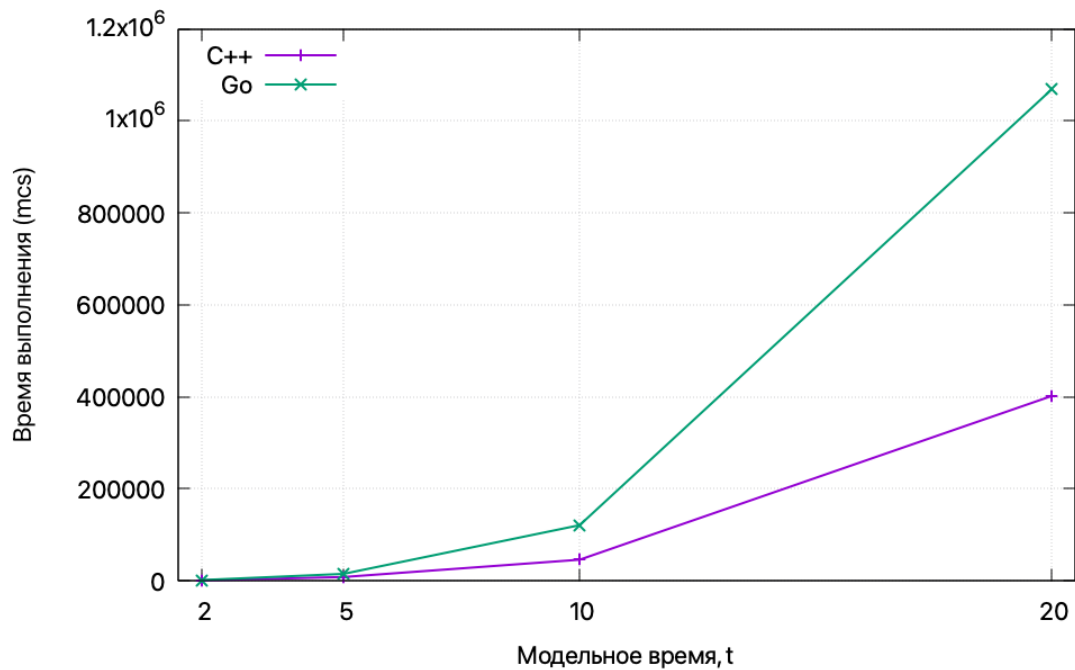


Рисунок 8 — Сравнение времени построения сетки для задачи Коши в Go- и C++-реализациях при разном модельном времени t_{max}

4.11.4 Сравнение ограничения числа узлов

В C++-реализации ограничение числа узлов на рассматриваемой функции почти не меняет время построения: без ограничения получено 369217 нс, при лимите 200 — 332264 нс, при лимите 500 — 361746 нс, при лимите 1000 — 357709 нс. Это означает, что в данной конфигурации сетка и без жёсткого лимита остаётся достаточно компактной, поэтому параметр N_{max} не становится главным фактором производительности.

В Go-реализации эффект выражен сильнее: без ограничения получено 4833921 нс/оп, при лимите 200 — 972821 нс/оп, при лимите 500 — 2160798 нс/оп, при лимите 1000 — 3104964 нс/оп. Следовательно, роль ограничения числа узлов зависит от реализации и конкретной функции: для Go этот параметр является сильным средством управления затратами, тогда как в C++ на данном сценарии его влияние существенно слабее.

4.12 Преимущества разработанного решения

На основе архитектуры проекта и экспериментальной программы можно выделить следующие преимущества разработанного решения:

- 1) модуль `compute` объединяет математическую компактность и практическую гибкость;
- 2) поддержка двух базисов позволяет исследовать баланс между локальной стоимостью и качеством аппроксимации;
- 3) механизм якорных точек расширяет базовый метод и устраняет важный класс ошибок адаптивного критерия;

- 4) параметр ограничения числа узлов даёт эффективный способ контролировать ресурсы;
- 5) поддержка двух режимов работы с дифференциальными уравнениями делает систему более универсальной;
- 6) экспериментальная программа позволяет анализировать поведение алгоритма в нескольких содержательных режимах, а не только на одном частном тесте;
- 7) сравнение Go- и C++-версий усиливает инженерную ценность работы и позволяет говорить не только о корректности алгоритма, но и о свойствах различных способов его реализации.

4.13 Ограничения текущей реализации

Несмотря на полученные результаты, текущая версия системы имеет и ряд ограничений.

- 1) Клиентская визуализация ориентирована прежде всего на двумерные задачи и не предоставляет развитых средств анализа для более высоких размерностей.
- 2) Выбор якорных точек производится вручную; автоматизированной стратегии их генерации пока не реализовано.
- 3) Для полного количественного анализа качества интерполяции требуется дальнейшее расширение набора тестовых функций и метрик ошибки.
- 4) В проекте пока отсутствует встроенная автоматическая генерация графиков по результатам бенчмарков, поэтому соответствующие иллюстрации подготавливаются отдельно.
- 5) Этап вычисления значения по готовой сетке требует дополнительной оптимизации из-за высокой стоимости рекурсивного обхода и большого числа повторных вычислений.
- 6) Для более строгого межъязыкового сопоставления в будущем полезно автоматизировать единый контур запуска и экспорта результатов Go- и C++-бенчмарков в один формат.

4.14 Иллюстрации двумерных функций, полученные во Flutter-клиенте

Для качественной оценки поведения алгоритма важно анализировать не только численные бенчмарки, но и визуальную структуру построенных сеток. Flutter-клиент позволяет наглядно увидеть отображение, задаваемое функцией и построенное по тем же данным, что используются в вычислительном модуле.

На рисунке 9 показана сетка и визуализация функции

$$u = x, \quad v = y (\sin(2.5x) + 0.35 \cos(6y) + 0.2 \sin(5x + 0.4) - 0.15 \cos(9y - 0.2)) \quad (53)$$

на области

$$x \in [0, 4], \quad y \in [0, 2] \quad (54)$$

при

$$\varepsilon = 0.001. \quad (55)$$

Такой пример удобен тем, что не содержит смешанных членов вида $\sin(xy)$, но при этом создаёт более содержательную структуру двумерной сетки. Это позволяет визуально увидеть, как адаптивная сетка концентрирует узлы в более сложных областях, не перегружая всю область равномерным дискретизационным шаблоном.

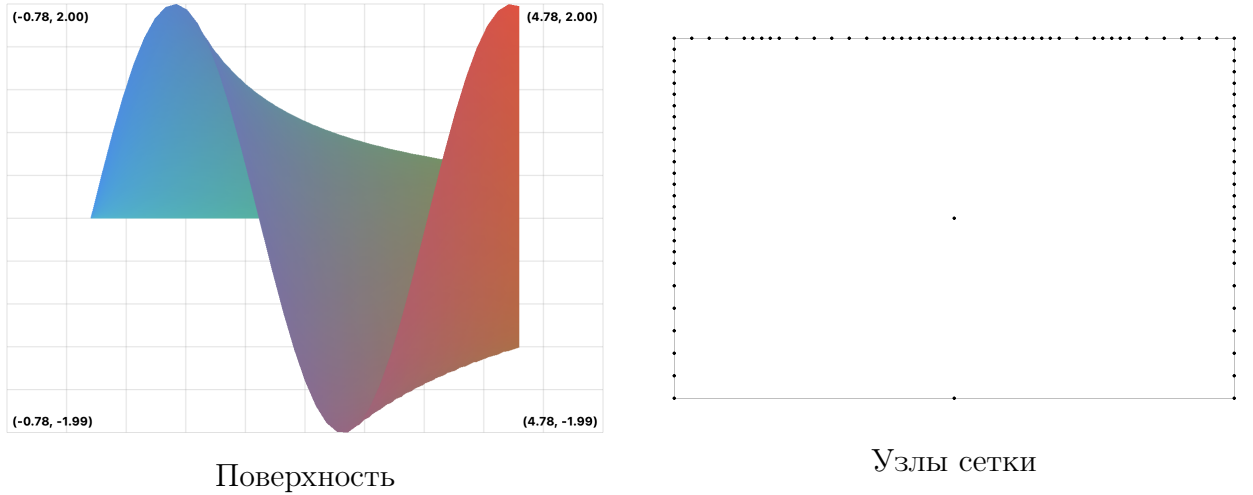


Рисунок 9 — Визуализация функции $u = x$, $v = y(\sin(2.5x) + 0.35 \cos(6y) + 0.2 \sin(5x + 0.4) - 0.15 \cos(9y - 0.2))$ во Flutter-клиенте и расположение узлов сетки

На рисунке 10 представлен пример визуализации динамической задачи, задаваемой системой

$$\frac{dx}{dt} = 0, \quad \frac{dy}{dt} = \sin(x) y + t \quad (56)$$

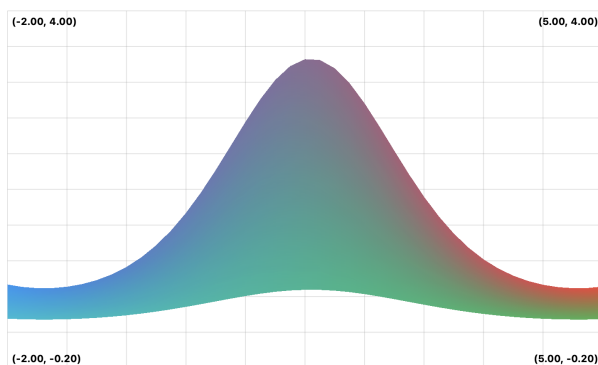
при начальных условиях

$$x \in [-2, 5], \quad y \in [0, 1] \quad (57)$$

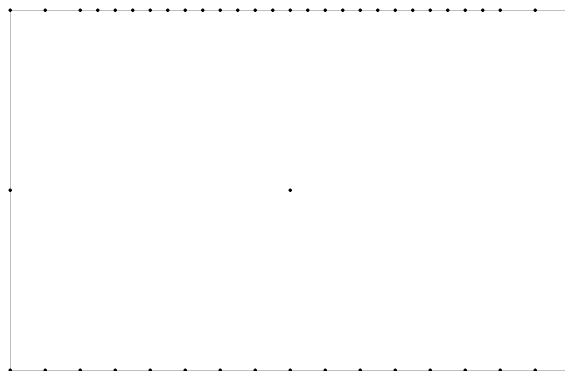
и при

$$\varepsilon = 0.001. \quad (58)$$

Этот пример особенно полезен для демонстрации того, что адаптивная интерполяция применима не только к аналитическим функциям, но и к отображениям, возникающим из численного интегрирования дифференциальных уравнений. На изображении можно увидеть, как локальная сложность динамики приводит к неоднородному распределению узлов.



Визуализация решения



Узлы сетки

Рисунок 10 — Визуализация дифференциального уравнения во Flutter-клиенте и расположение узлов сетки

4.15 Перспективы дальнейшего развития

Дальнейшее развитие проекта может идти по нескольким направлениям:

- 1) автоматизация выбора якорных точек на основе анализа локальной структуры ошибки;
- 2) расширение визуализации на многомерные случаи;
- 3) добавление встроенного контура сохранения результатов бенчмарков и генерации графиков;
- 4) расширение библиотеки тестовых сценариев для количественного исследования норм ошибки;
- 5) оптимизация рекурсивного этапа вычисления значения по готовой сетке;
- 6) исследование комбинированных стратегий выбора базиса и ограничения числа узлов в зависимости от характера функции;
- 7) разработка единого автоматизированного режима сравнения Go- и C++-версий библиотеки.

Выводы по главе

В четвёртой главе был проведён развёрнутый анализ результатов дипломной работы с акцентом на экспериментальную программу и производительность вычислительного модуля `compute`. Показано, что использование взвешенного по уровню критерия остановки позволяет строить сетки для функций со смешанной зависимостью координат без обязательного задания жёстких ограничений глубины. Измерения для Go-реализации подтвердили устойчивое преимущество квадратичного базиса, заметный выигрыш параллельного режима на нетривиальных двумерных функциях и практическую значимость ограничения числа узлов как средства управления затратами. Для двумерной динамической системы установлено, что итеративный подход эффективнее

по времени на всём рассмотренном диапазоне горизонтов за счёт меньшей размерности и меньшего числа узлов, однако он даёт значения только в целочисленных узлах модельного времени, тогда как для произвольного момента t необходим подход с t как дополнительной координатой. Дополнительное сопоставление Go- и C++-реализаций на задаче Коши показало, что C++-версия быстрее на всём диапазоне модельного времени, причём разрыв растёт с увеличением t_{max} , что объясняется доминированием стоимости самих вычислений над накладными расходами на параллелизм. В совокупности полученные результаты подтверждают достижение цели дипломной работы и создают основу для дальнейшего развития проекта.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы была решена задача реализации и исследования параллельных алгоритмов интерполяции на основе разреженных сеток с квадратичным базисом.

Основным результатом работы стал вычислительный модуль `compute`, в котором реализованы:

- 1) построение адаптивной разреженной сетки для скалярных и векторных отображений;
- 2) поддержка линейного и квадратичного базисов;
- 3) последовательный и параллельный режимы построения;
- 4) рекурсивное вычисление значения интерполанта;
- 5) композиция отображений и итеративный режим через `MakeNextIteration`;
- 6) механизм якорных точек для предотвращения ложной остановки адаптивного алгоритма;
- 7) параметр жёсткого ограничения числа узлов как средство дополнительной оптимизации.

В ходе работы было показано, что разреженные сетки представляют собой эффективный подход к аппроксимации отображений и операторов перехода, возникающих при численном моделировании динамических систем. Отдельно было исследовано применение метода к дифференциальным уравнениям в двух режимах: через включение времени как интервальной неопределённости и через последовательное построение одношагового оператора.

Особое значение имеет предложенный в работе механизм якорных точек. На примере функции $\sin(x)$ на интервале $[0, 4\pi]$ было показано, что базовый адаптивный критерий может ошибочно интерпретировать функцию как почти линейную, если опорные точки попадают в нули. Реализованный механизм якорных точек позволяет в ручном режиме указать важные области и принудить алгоритм к дополнительному уточнению. Это расширение не было заложено в исходной статье [6] и является одним из существенных авторских результатов дипломного проекта.

Экспериментальная часть работы показала важность сравнительного анализа различных режимов использования модуля. Были рассмотрены:

- 1) сравнение линейного и квадратичного базисов;
- 2) сравнение последовательного и параллельного построения;
- 3) влияние размерности задачи и размера области на время построения;
- 4) влияние якорных точек на корректность и стоимость адаптивного построения;
- 5) сравнение двух способов работы с дифференциальными уравнениями;

б) влияние ограничения числа узлов на вычислительные затраты.

Тем самым дипломная работа привела не только к созданию программной реализации, но и к построению содержательной экспериментальной базы для анализа производительности и границ применимости метода.

Цель работы — реализация параллельных алгоритмов интерполяции на основе разреженных сеток с квадратичным базисом — достигнута. Поставленные задачи выполнены в полном объёме.

Практическая значимость работы состоит в том, что созданный вычислительный модуль может использоваться:

- 1) для численного эксперимента с многомерными отображениями;
- 2) для исследования операторов перехода динамических систем;
- 3) для сравнительного анализа различных стратегий адаптивной интерполяции;
- 4) как основа для дальнейшего развития систем визуализации и прикладных сервисов научных вычислений.

Серверный слой на gRPC и клиент визуализации на Flutter расширяют область применения проекта, однако центральным содержанием дипломной работы является именно модуль `compute`, в котором сосредоточены ключевые алгоритмические результаты.

Перспективы дальнейшего развития проекта связаны с автоматизацией выбора якорных точек, дальнейшим исследованием ошибок интерполяции, расширением визуализации на многомерные задачи и построением более развитого контура автоматического анализа результатов бенчмарков.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Bungartz H.-J., Griebel M. Sparse Grids. — Acta Numerica, 2004.
2. Zenger C. Sparse Grids // Parallel Algorithms for Partial Differential Equations. — 1991. — С. 241—251.
3. Quarteroni A., Sacco R., Saleri F. Numerical Mathematics. — Berlin : Springer, 2007.
4. Hairer E., Norsett S. P., Wanner G. Solving Ordinary Differential Equations I: Nonstiff Problems. — Berlin : Springer, 1993.
5. Stoer J., Bulirsch R. Introduction to Numerical Analysis. — New York : Springer, 2002.
6. Морозов А. Ю. Моделирование динамических систем с интервальными параметрами [Электронный ресурс]. — 2024. — URL: https://www.frccsc.ru/sites/default/files/docs/ds/002-073-04/diss/37-morozov/ds04-37-morozov_main.pdf?37 (дата обр. 01.05.2026).
7. Moin P. Fundamentals of Engineering Numerical Analysis. — Cambridge : Cambridge University Press, 2010.
8. The Go Programming Language Documentation [Электронный ресурс]. — URL: <https://go.dev/doc/> (дата обр. 01.05.2026).
9. Expr Language Definition and Documentation [Электронный ресурс]. — URL: <https://expr-lang.org/> (дата обр. 01.05.2026).
10. Flutter Documentation [Электронный ресурс]. — URL: <https://docs.flutter.dev/> (дата обр. 01.05.2026).

ПРИЛОЖЕНИЕ А

Репозиторий с исходным кодом

В рамках выполнения выпускной квалификационной работы был разработан программный комплекс для интерполяции отображений и решений дифференциальных уравнений на основе адаптивных разреженных сеток с поддержкой квадратичного базиса и параллельного построения.

Полный исходный код проекта, включая вычислительный модуль, серверный слой, клиент визуализации и вспомогательные средства развертывания, размещён в открытом репозитории на платформе GitHub.

Ссылка на репозиторий: <https://github.com/DesCaldnd/Diplom>

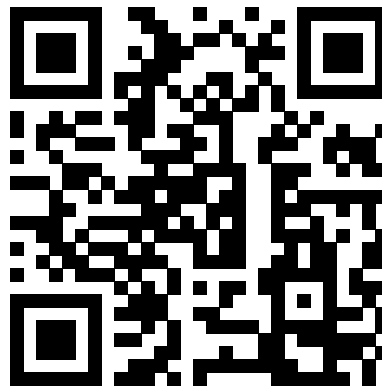


Рисунок А.1 — QR-код со ссылкой на репозиторий с исходным кодом проекта

В состав репозитория входят:

- 1) исходный код вычислительного модуля `compute`, реализующего адаптивные разреженные сетки;
- 2) gRPC-сервис, обеспечивающий удалённый вызов построения сетки и вычисления отображений;
- 3) клиентское приложение визуализации на Flutter;
- 4) protobuf-описание контракта обмена данными;
- 5) набор тестов, примеров и бенчмарков для исследования корректности и производительности алгоритмов.